

Procedurální modelování rostlinných struktur

Martin Sedmera

13. února 2020

Obsah

Úvod	7
1 Základy lineární algebry	9
1.1 Vektorový prostor	9
1.1.1 Číselné těleso	9
1.1.2 Sčítání $\oplus$ a násobení $\otimes$ vektorů	10
1.1.3 Příklady vektorových prostorů	11
1.1.3.1 Prostor uspořádaných n -tic	11
1.1.3.2 Prostor matic	12
1.2 Lineární závislost	13
1.2.1 Soubor vektorů	13
1.2.1.1 Vlastnosti součtu souborů	14
1.2.2 Lineární kombinace	14
1.2.3 Lineární obal	15
1.2.4 Lineární závislost a nezávislost	16
1.2.5 Báze a dimenze vektorového prostoru	17
1.2.6 Souřadnice vektoru v bázi	17
1.3 Lineární zobrazení	18
1.4 Matice lineárního zobrazení	20
1.4.1 Matice lineárního zobrazení pro obecné vektorové pro-	
story	21
1.4.1.1 Speciální případ pro 2 obecné shodné prostory	22
1.4.2 Rotace vektorů ve 2D	22
1.4.2.1 Matice rotace	22
1.4.2.2 Součtové vzorce pro goniometrické funkce . .	24
2 Generování objektů pomocí gramatik	25
2.1 L-systémy	25
2.1.1 D0L-systémy	25

2.2	Interpretace D0L-systémů pomocí takzvané želví grafiky . . .	27
2.2.1	Větvený D0L-systém	27
2.3	Želví grafika ve 3D	28
2.3.1	Rotace želvy v prostoru	29
2.3.2	Pohyb želvy vpřed v prostoru	31
2.4	Pokročilejší metody pro generování rostlin	31
3	Implementace	33
3.1	Implementace přepisovacího algoritmu	33
3.2	Implementace želví grafiky	34
3.2.1	Vstup	34
3.2.2	Želva	35
3.2.2.1	Reprezentace želvy	35
3.2.2.2	Rotace	35
3.2.2.3	Implementace matic	36
3.2.2.4	Implementace pohybu vpřed	37
3.2.2.5	Implementace větví	38
3.2.3	Hlavní program	39
3.2.3.1	Implementace různých délek a šířek kroku vpřed	39
3.3	3D grafický výstup	39
3.3.1	Kamera	40
3.3.1.1	Otáčející se kamera	40
3.3.2	Kreslení čar	40
3.3.3	Osvětlení	41
4	Výsledky	42
4.1	Dvojměrná větvená struktura	42
4.2	Větvená struktura se změnou velikosti úhlu	42
4.3	Strom s proměnlivou šířkou čáry	42
4.4	3D keř	44
4.5	Větší 3D rostlina	44
4.6	Kapradí	46
	Závěr	48
	Přílohy	50
	Literatura	55

Úvod

Procedurální modelování je jeden ze způsobů modelování objektů. Pomocí této metody se dají snadno modelovat objekty, které se řídí nějakými pravidly, například fraktály nebo rostliny. V této práci se seznámíte s průběhem procesu procedurálního modelování, jak funguje a s mým vlastním přístupem k této problematice.

Práce je členěna do několika částí, reprezentovaných kapitolami tohoto textu. Každá má vlastní cíl a smysl.

V první kapitole je definován od základů matematický aparát, který je využit v 2. části práce. Jedná se převážně o základy lineární algebry, proto jako zdroj sloužila skripta k lineární algebře od Jiřího Pytlíčka[3]. Cílem této kapitoly je poskytnutí relevantních matematických základů pro zbytek práce srozumitelnou formou a naučit se psát formální matematické vzorce a důkazy na počítači.

V druhé kapitole je vysvětleno procedurální generování rostlin založené na grafické reprezentaci řetězců znaků. Použité řetězce vznikají pomocí formálních gramatik. V této práci jsou k generování řetězců používány takzvané D0L-systémy - deterministické bezkontextové L-systémy. Více o D0L-systémech, i o tom, jak funguje jejich grafická interpretace, se dočtete v této kapitole. Tato část práce je převážně kompilačního charakteru, není postavena na vlastním výzkumu. Jako zdroj slouží především „bible“ oboru počítačového generování rostlin „Algorithmic Beauty of Plants“ [2] a další publikace od nejváženějšího vědce v oboru Przemyslaw Prusikiewicz[8]. Pochopení těchto principů a jejich srozumitelné prezentování je důležité pro další cíl práce: funkční vlastní implementace procedurálního modelování rostlin pomocí D0L-systémů.

Třetí kapitola popisuje mojí vlastní implementaci D0L-systémů popsaných ve 2. kapitole pomocí programovacího jazyka Python. V této části práce je představen vlastní kód a přístup a jsou popsány zajímavější problémy, které nastaly při práci na programu. Tato kapitola je pro celou práci stěžejní a odvíjí se od ní všechny ostatní. Hlavní cíl je vytvořit funkční program a

pochopit, na jakých principech funguje.

Ve 4. kapitole naleznete ukázky rostlin, které vznikly pomocí programu popsaného ve 3. kapitole. Protože vytváření vlastních modelů není jednoduché a spadá spíše do biologie, představuji některé rostliny z jiných zdrojů [2, 4, 5]. V této kapitole je představeno i několik výsledků vlastní experimentace.

Kapitola 1

Základy lineární algebry

V této kapitole představuji matematický aparát sloužící jako základ pro další část práce. Zde uvedené vzorce, teorémy, důkazy atd... nejsou mými vlastními myšlenkami, ani vlastními objevy. Vše v této kapitole je citováno z [3], ale opatřeno vlastními komentáři. Dále jsou vybrána pouze témata relevantní pro kontext této práce. Cílem této části práce je pochopení uvedeného aparátu a dovednost psaní matematických vzorců na počítači.

1.1 Vektorový prostor

1.1.1 Číselné těleso

Definice 1. Množina $T \subset \mathcal{C}$ se nazývá **číselným tělesem**, právě když platí:

1. $\{0, 1\} \in T$
2. $(\forall \alpha \in T)(\forall \beta \in T)(\alpha + \beta \in T)$
3. $(\forall \alpha \in T)(\forall \beta \in T)(\alpha \beta \in T)$
4. $(\forall \alpha \in T)(-\alpha \in T)$
5. $(\forall \alpha \in T)(\alpha \neq 0)(1/\alpha \in T)$

Výraz $T \subset \mathcal{C}$ značí, že **číselné** těleso T je podmnožinou **komplexních** čísel. 2. podmínka definuje uzavřenost číselného tělesa T vůči sčítání, 3. proti násobení, 4. obrácené hodnotě - z čehož vyplývá, že $\mathcal{N}$ (celá čísla) není číselné těleso, a 5. podmínka vůči převrácené hodnotě, která vylučuje celá čísla $\mathcal{Z}$ jako číselná tělesa. Uzavřenost množiny znamená, že výsledky těchto operací budou součástí téže množiny.

1.1.2 Sčítání $\oplus$ a násobení $\otimes$ vektorů

Definice 2. **Zobrazení** je předpis, který prvku jedné množiny, **definičního oboru**, přiřadí nanejvýš jeden prvek další množiny, **oboru hodnot**.

Definice 3. **Kartézský součin** $A \times B = \{(\vec{a}, \vec{b}) | \vec{a} \in A \wedge \vec{b} \in B\}$ je množina **uspořádaných dvojic** $(\vec{a}, \vec{b})$, kdy $\vec{a}$ je prvek množiny A a $\vec{b}$ je prvek množiny B . **Zobrazení** $f : A \mapsto B$ je taková podmnožina **kartézského součinu** $A \times B$, pro kterou platí: $(\forall \vec{a} \in A)(\exists_1 \vec{b} \in B)((\vec{a}, \vec{b}) \in f)$.

Definice 4. Sčítání a násobení vektorů

Nechť máme:

1. číselné těleso T
2. neprázdnou množinu V
3. zobrazení $\oplus : V \times V \mapsto V$,
4. zobrazení $\otimes : T \times V \mapsto V$,

Prvky množiny V nazýváme vektory, operaci $\oplus$ nazýváme sčítání vektorů, operaci $\otimes$ nazýváme násobení vektorů číslem z tělesa. Dále v textu budeme značit sčítání 2 vektorů symbolem $+$. $(\forall \vec{a}, \vec{b} \in V)(\vec{a} + \vec{b} := \vec{a} \oplus \vec{b})$. Stejně tak násobení vektoru budeme značit $(\forall \alpha \in T)(\forall \vec{a} \in V)(\alpha \vec{a} := \alpha \otimes \vec{a})$.

Definice 5. Množinu V nazýváme **vektorovým prostorem nad tělesem** T , právě když platí:

1. $(\forall \vec{a} \in V)(\forall \vec{b} \in V)(\vec{a} + \vec{b} = \vec{b} + \vec{a})$

Tomuto se říká komutativní zákon pro sčítání vektorů $(+)$, který je nám dobře znám z čísel.

2. $(\forall \vec{a} \in V)(\forall \vec{b} \in V)(\forall \vec{c} \in V)(\vec{a} + (\vec{b} + \vec{c}) = (\vec{a} + \vec{b}) + \vec{c})$

Takto zní asociativní zákon pro sčítání vektorů.

3. $(\exists \theta \in V)(\forall \vec{a} \in V)((\vec{a} + \theta = \vec{a}))$

Tímto způsobem definujeme **nulový vektor**, který je definován jako vektor, který když přičteme k jakémukoli vektoru, dostaneme tentýž vektor.

4. $(\forall \vec{a} \in V)(\exists \vec{b} \in V)(\vec{a} + \vec{b} = \theta)$

Toto definuje **opačný vektor** k vektoru $\vec{a}$. Pro jeho definici používáme výše definovaný **nulový vektor**. Značíme ho $-\vec{a}$.

$$5. (\forall \alpha \in T)(\forall \beta \in T)(\forall \vec{a} \in V)((\alpha(\beta \vec{a}) = (\alpha\beta)\vec{a})$$

Toto je asociativní zákon pro násobení vektorů.

$$6. (\forall \vec{a} \in V)(1 \times \vec{a} = \vec{a})$$

$$7. (\forall \alpha \in T)(\forall \beta \in T)(\forall \vec{a} \in V)((\alpha + \beta)\vec{a} = (\alpha\vec{a}) + (\beta\vec{a}))$$

Toto definuje distributivu vzhledem ke sčítání čísel, a tento princip známe z násobení čísel. Všimněte si, že ve výrazu $(\alpha + \beta)$ jde o sčítání čísel.

$$8. (\forall \alpha \in T)(\forall \vec{a} \in V)(\forall \vec{b} \in V)(\alpha(\vec{a} + \vec{b}) = (\alpha\vec{a}) + (\alpha\vec{b}))$$

V tomto případě jde o sčítání vektorů, nikoliv čísel. Jednoduše to poznáme tak, že vektory mají nad sebou šipku $\vec{}$ a skaláry (čísla) jsou značena řeckými písmeny.

Poznámka 6. Všechny tyto základní principy opět zavádíme, protože se nyní pohybujeme v operacích s vektory, nikoliv s čísly. Bez definování těchto základů nemůžeme pokročit ke složitějším tématům.

Pro **vektorový prostor** V nad číselným tělesem T platí podle výše uvedené definice následující:

1. Ve V existuje právě 1 **nulový vektor** θ .
2. Ke každému vektoru z V existuje právě 1 **vektor opačný**.
3. Pro každé $(\vec{a}, \vec{b}) \in V \times V$ má rovnice $\vec{a} = \vec{b} + \vec{x}$ právě 1 řešení: $\vec{x} = -\vec{b} + \vec{a}$.
4. $(\forall \alpha \in T)(\alpha\theta = \theta)$,
 $(\forall \vec{a} \in V)(0\vec{a} = \theta)$.
5. $(\forall \alpha \in T)(\forall \vec{a} \in V)(\alpha\vec{a} = 0 \Rightarrow (\alpha = 0 \vee \vec{a} = \theta))$.
6. $(\forall \alpha \in T)(\forall \vec{a} \in V)((-\alpha)\vec{a} = -(\alpha\vec{a}) = \alpha(-\vec{a}))$.

1.1.3 Příklady vektorových prostorů

Pro následující příklady mějme **číselné těleso** T .

1.1.3.1 Prostor uspořádaných n -tic

Prvním příkladem je prostor $T^n, n \in \{1, 2, 3, \dots, n\}$. Prvky jsou uspořádané n -tice čísel z tělesa T : $\vec{a} = (\alpha_1, \dots, \alpha_n)$. Pro $(\forall i \in T)(\alpha_i \in T)$ a čísla α_i nazýváme **složky vektoru** $\vec{a}$. Operace sčítání vektorů $(+)$ a násobení vektorů provádíme po složkách. Mějme $\vec{a} = (\alpha_1, \dots, \alpha_n)$ a $\vec{b} = (\beta_1, \dots, \beta_n)$, pak $\vec{a} + \vec{b} = (\alpha_1 + \beta_1, \alpha_2 + \beta_2, \dots, \alpha_n + \beta_n)$.

1.1.3.2 Prostor matic

Dalším příkladem vektorového prostoru je prostor $T^{m,n}$, $m, n \in \mathcal{N}$. Jeho prvky jsou **matice**:

$$\mathbb{A} = \begin{pmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \dots & \dots & \alpha_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ \alpha_{m1} & \dots & \dots & \alpha_{mn} \end{pmatrix}.$$

Matice je čtvercové, či obdelníkové schéma **prvků matice** o m řádcích a n sloupcích. V takovém případě hovoříme o matici rozměrů $m \times n$. Matice značíme velkými tučnými písmeny $\mathbb{A}, \mathbb{B}, \mathbb{C} \dots$. Čísla α_{ij} kdy $i \in \{1, 2, \dots, m\}$ a $j \in \{1, 2, \dots, n\}$ nazýváme elementy matice. Čísla z tělesa $\alpha_1, \dots, \alpha_m$ budeme nazývat **složky vektoru**. **Nulový vektor** má všechny složky nulové. Konkrétně prvky prostoru T^{m1} nazveme **sloupcové vektory**:

$$\vec{a} = \begin{pmatrix} \alpha_{11} \\ \alpha_{21} \\ \vdots \\ \alpha_{m1} \end{pmatrix} = \begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_m \end{pmatrix}.$$

Definujeme 2 operace s maticemi:

- **Sčítání matic:** Mějme dvě matice stejných rozměrů $m \times n$ $\mathbb{A}, \mathbb{B} \in T^{m,n}$ a prvky $\alpha_{ij} \in \mathbb{A}, \beta_{ij} \in \mathbb{B}$.

$$\mathbb{A} + \mathbb{B} = \begin{pmatrix} \alpha_{11} + \beta_{11} & \alpha_{12} + \beta_{12} & \dots & \alpha_{1n} + \beta_{1n} \\ \alpha_{21} + \beta_{21} & \alpha_{22} + \beta_{22} & \dots & \alpha_{2n} + \beta_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ \alpha_{m1} + \beta_{m1} & \dots & \dots & \alpha_{mn} + \beta_{mn} \end{pmatrix}.$$

Sčítání matic je komutativní.

- **Násobení dvou matic:** Definujeme operaci násobení dvou matic. Tuto operaci budeme využívat při rotaci vektorů. Jedinou podmínkou násobení matic je, že počet sloupců první matice je stejný, jako počet řádků druhé. Nechť $\mathbb{A} \in T^{m,s}$ a $\mathbb{B} \in T^{s,n}$, pak $\mathbb{A}\mathbb{B} \in T^{m,n}$ a i, j -tý prvek součinu, který značíme $\mathbb{A}\mathbb{B}$, se vypočítá jako

$$(\mathbb{A}\mathbb{B})_{ij} = \sum_{\ell=1}^s \alpha_{i\ell} \beta_{\ell j}. \quad (1.1)$$

Lidsky řečeno, to znamená, že prvek na souřadnicích $[1; 1]$ výsledné matice je součet součinů prvků z 1. řádku matice $\mathbb{A}$ s prvky 1. sloupce matice $\mathbb{B}$, kdy vždy násobíme první prvek s prvním, druhý s druhým, atd... Například pro součin dvou čtvercových matic rozměru 2×2 bude výsledná matice vypadat takto:

$$\mathbb{A} = \begin{pmatrix} \alpha_{11} & \alpha_{12} \\ \alpha_{21} & \alpha_{22} \end{pmatrix}, \mathbb{B} = \begin{pmatrix} \beta_{11} & \beta_{12} \\ \beta_{21} & \beta_{22} \end{pmatrix}$$

$$\mathbb{A}\mathbb{B} = \begin{pmatrix} \alpha_{11}\beta_{11} + \alpha_{12}\beta_{21} & \alpha_{11}\beta_{12} + \alpha_{12}\beta_{22} \\ \alpha_{21}\beta_{11} + \alpha_{22}\beta_{21} & \alpha_{21}\beta_{12} + \alpha_{22}\beta_{22} \end{pmatrix}$$

Násobení matic není komutativní, $\mathbb{A}\mathbb{B} \neq \mathbb{B}\mathbb{A}$. Jak bylo výše zmíněno, podmínkou násobení matic je stejný počet sloupců matice $\mathbb{A}$, jako počet řádků matice $\mathbb{B}$. Jaký tvar bude tedy mít výsledek násobení dvou matic různého tvaru? Bude mít tolik řádků, kolik $\mathbb{A}$ má řádků, a tolik sloupců, kolik sloupců má $\mathbb{B}$.

1.2 Lineární závislost

1.2.1 Soubor vektorů

Bud' $n \in \mathcal{N}$. Zavedeme $(\nu \subset \mathcal{N})(\nu := \{1, 2, 3, \dots, n\})$

Bud' V **vektorovým prostorem** nad **číselným tělesem** T . **Soubor vektorů** délky $n \in \nu$ je uspořádaná n -tice $(\vec{x}_1, \dots, \vec{x}_n)$ pro kterou platí: $(\forall i \in \nu)(\vec{x}_i \in V)$.

Bud' $(\vec{x}_1, \dots, \vec{x}_n)$ **soubor vektorů** z V .

Definice 7. Součet souboru $(\vec{x}_1, \dots, \vec{x}_n)$ značíme

$$\sum_{i=1}^n \vec{x}_i = (\vec{x}_1 + \vec{x}_2 + \dots + \vec{x}_n).$$

Definice 8. Prázdný součet (pro $j \in \mathcal{Z}$):

$$\sum_{i=j}^{j-1} \vec{x}_i = \theta$$

Jednoduše řečeno: sčítáme prvky prázdné množiny. Například pro $j = 5$ sčítáme prvky od 5. prvku po 4. prvek souboru.

1.2.1.1 Vlastnosti součtu souborů

Věta 9. *Součet souboru má následující vlastnosti:*

1. Zobecněný asociativní zákon:

$$\sum_{j=1}^n \vec{x}_j = \sum_{j=1}^k \vec{x}_j + \sum_{j=k+1}^n \vec{x}_j \quad (\forall k \in \nu).$$

2. Zobecněný komutativní zákon: Buď $(k_1, \dots, k_n)$ permutace množiny ν , pak

$$\sum_{i=1}^n \vec{x}_i = \sum_{i=1}^n \vec{x}_{k_i}$$

Permutace n -prvkové množiny je uspořádaná n -tice, která obsahuje každý prvek právě jednou. Tento zákon nám tedy říká, že **součet souboru** bude stejný, bez ohledu na pořadí ve kterém sčítáme prvky souboru.

3. Možnost “vytknout” skalár před součet souboru

$$(\forall \alpha \in T) \left(\alpha \sum_{i=1}^n \vec{x}_i = \sum_{i=1}^n (\alpha \vec{x}_i) \right)$$

4. Mějme 2 soubory vektorů $(\vec{x}_1, \dots, \vec{x}_n)$ a $(\vec{y}_1, \dots, \vec{y}_n)$ z V o stejné délce $n \in \nu$.

$$\sum_{i=1}^n \vec{x}_i + \sum_{i=1}^n \vec{y}_i = \sum_{i=1}^n (\vec{x}_i + \vec{y}_i)$$

1.2.2 Lineární kombinace

Definice 10. Necht $(\vec{x}_1, \dots, \vec{x}_n) \in V^n$ a $(\alpha_1, \dots, \alpha_n) \in T^n$. Potom vektor

$$\vec{x} = \sum_{i=1}^n \alpha_i \vec{x}_i$$

nazýváme **lineární kombinací vektorů (souboru)** $(\vec{x}_1, \dots, \vec{x}_n)$. Čísla $(\alpha_1, \dots, \alpha_n)$ nazýváme **koefficienty lineární kombinace**. Lineární kombinace se nazývá **triviální** $\iff (\forall i \in \nu)(\alpha_i = 0)$. V opačném případě se nazývá **netriviální lineární kombinace**.

1.2.3 Lineární obal

Definice 11. Buď $(\vec{x}_1, \dots, \vec{x}_n)$ soubor vektorů z V . Množinu všech lineárních kombinací souboru $(\vec{x}_1, \dots, \vec{x}_n)$ nazýváme **lineární obal souboru** a značíme ho $[\vec{x}_1, \dots, \vec{x}_n]_\lambda$, tj.

$$[\vec{x}_1, \dots, \vec{x}_n]_\lambda = \left\{ \sum_{i=1}^n \alpha_i \vec{x}_i \mid (\alpha_1, \dots, \alpha_n) \in T^n \right\}.$$

Věta 12. *Vlastnosti lineárního obalu*

Nechť $(\vec{x}_1, \dots, \vec{x}_n)$ je soubor vektorů z V . Potom platí:

1. $\theta \in [\vec{x}_1, \dots, \vec{x}_n]$

Nulový vektor je elementem **lineárního obalu** souboru, protože nulový vektor θ je výsledek triviální kombinace (viz definice 10).

2. $\vec{x}_{n+1} \in [\vec{x}_1, \dots, \vec{x}_n]_\lambda \implies [\vec{x}_1, \dots, \vec{x}_n]_\lambda = [\vec{x}_1, \dots, \vec{x}_{n+1}]_\lambda$

Důkaz je pomocí dvou inkluzí.

- (a) Chceme dokázat $[\vec{x}_1, \dots, \vec{x}_n]_\lambda \subset [\vec{x}_1, \dots, \vec{x}_{n+1}]_\lambda$. Zvolme libovolné $\vec{x} \in [\vec{x}_1, \dots, \vec{x}_n]_\lambda$, potom víme, že $\exists (\alpha_1, \dots, \alpha_n)$ takové, že

$$\vec{x} = \sum_{i=1}^n \alpha_i \vec{x}_i = \sum_{i=1}^n \alpha_i \vec{x}_i + 0 \vec{x}_{n+1}.$$

To znamená, že $\vec{x}$ lze napsat jako **lineární kombinaci** $\vec{x}_1, \dots, \vec{x}_n, \vec{x}_{n+1}$.

Z toho plyne $\vec{x} \in [\vec{x}_1, \dots, \vec{x}_{n+1}]_\lambda$.

- (b) Nyní musíme dokázat, že $[\vec{x}_1, \dots, \vec{x}_{n+1}]_\lambda \subset [\vec{x}_1, \dots, \vec{x}_n]_\lambda$:

Zvolme libovolné $\vec{x} \in [\vec{x}_1, \dots, \vec{x}_{n+1}]_\lambda$. Existuje tedy $(\alpha_1, \dots, \alpha_{n+1}) \in T^{n+1}$, která splňuje

$$\vec{x} = \sum_{i=1}^{n+1} \alpha_i \vec{x}_i = \sum_{i=1}^n \alpha_i \vec{x}_i + \alpha_{n+1} \vec{x}_{n+1}. \quad (1.2)$$

Víme, že $\vec{x}_{n+1} \in [\vec{x}_1, \dots, \vec{x}_n]_\lambda$, tudíž $\exists (\beta_1, \dots, \beta_n) \in T^n$, pro které platí

$$\vec{x}_{n+1} = \sum_{i=1}^n \beta_i \vec{x}_i.$$

Dosazením do rovnice (1.2) dostaneme

$$\vec{x} = \sum_{i=1}^n \alpha_i \vec{x}_i + \alpha_{n+1} \sum_{i=1}^n \beta_i \vec{x}_i,$$

což můžeme zjednodušit na

$$\vec{x} = \sum_{i=1}^n (\alpha_i + \alpha_{n+1} \beta_i) \vec{x}_i.$$

Jelikož můžeme vyjádřit $\vec{x}$ jako lineární kombinaci $(\vec{x}_1, \dots, \vec{x}_n)$, tak jsme dokázali:

$$\vec{x} \in [\vec{x}_1, \dots, \vec{x}_n]_{\lambda}.$$

3. Buď $(k_1, \dots, k_n)$ permutace množiny $\mathcal{N}$, potom: $[\vec{x}_1, \dots, \vec{x}_n]_{\lambda} = [\vec{x}_{k_1}, \dots, \vec{x}_{k_n}]_{\lambda}$.

Toto plyne ze zobecněného kumulativního zákona $\sum_{i=1}^n \vec{x}_i = \sum_{i=1}^n \vec{x}_{k_i}$ (viz 9).

4. Mějme $\vec{x} \in [\vec{x}_1, \dots, \vec{x}_n]_{\lambda}, \vec{y} \in [\vec{x}_1, \dots, \vec{x}_n]_{\lambda}, \alpha \in T$, potom $\vec{x} + \vec{y} \in [\vec{x}_1, \dots, \vec{x}_n]_{\lambda}$ a $\alpha \vec{x} \in [\vec{x}_1, \dots, \vec{x}_n]_{\lambda}$. Tyto vztahy představují uzavřenost **lineárního obalu** proti sčítání vektorů a násobení vektoru skalárem. Z toho vyplývá, že $[\vec{x}_1, \dots, \vec{x}_n]_{\lambda}$ je vektorový prostor.

1.2.4 Lineární závislost a nezávislost

Definice 13. Nechť je $(\vec{x}_1, \dots, \vec{x}_n)$ soubor vektorů z V . Soubor $(\vec{x}_1, \dots, \vec{x}_n)$ je **lineárně závislý (značeno LZ)**, právě když existuje **netriviální lineární kombinace**, jejíž výsledek je roven θ . Soubor je **lineárně nezávislý (LN)**, právě když není **lineárně závislý**.

Alternativní zápis:

$$(\vec{x}_1, \dots, \vec{x}_n) \text{ je LN} \iff \exists (\alpha_1, \dots, \alpha_n) \in T \left(\sum_{i=1}^n \alpha_i \vec{x}_i = \theta \wedge \sum_{i=1}^n |\alpha_i| > 0 \right)$$

Soubor $(\vec{x}_1, \dots, \vec{x}_n)$ je LN $\iff$ není LZ.

Definice 14. Soubor $(\vec{x}_1, \dots, \vec{x}_n)$ **konečně generuje vektorový prostor** V , právě když platí $[\vec{x}_1, \dots, \vec{x}_n]_{\lambda} = V$. V takovém případě označíme soubor $(\vec{x}_1, \dots, \vec{x}_n)$ jako **generující soubor** prostoru V a jednotlivé vektory $\vec{x}_i (i \in \nu)$ nazveme **generátory** prostoru V .

1.2.5 Báze a dimenze vektorového prostoru

Definice 15. Existuje-li soubor **generátorů** $(\vec{x}_1, \dots, \vec{x}_n)$, který je LN a zároveň **generuje** vektorový prostor V , tak ho nazýváme **konečnou bází** prostoru V .

Dimenze prostoru V je nejvyšší počet prvků LN souboru v prostoru V .

Značíme $\dim V = n$. Z tohoto vyplývá, že pro $\dim V = n$ existuje ve V soubor $(\vec{x}_1, \dots, \vec{x}_n)$ délky n , ale každý soubor o délce $n+1$ $(\vec{x}_1, \dots, \vec{x}_{n+1})$ bude LZ. Každý prostor V^n uspořádaných n -tic má $\dim V = n$. Toto dokážeme konstrukcí takzvané **standartní báze**, což je báze, která vypadá následovně:

$$\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \vec{e}_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, \vec{e}_{n-1} = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ 0 \end{pmatrix}, \vec{e}_n = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}.$$

Je to **báze**, jelikož každý další vektor by již byl LZ a zároveň platí $[(\vec{e}_1, \vec{e}_2, \dots, \vec{e}_n)]_\lambda = V$.

1.2.6 Souřadnice vektoru v bázi

Nechť $\mathcal{X} = (\vec{x}_1, \dots, \vec{x}_n)$ je **báze** prostoru V . Potom platí

$$\forall \vec{v} \in V : \exists (\alpha_1, \dots, \alpha_n) \in T; \left(\vec{v} = \sum_{i=1}^n \alpha_i \vec{x}_i \right).$$

Tento vztah znamená, že můžeme libovolný vektor z prostoru V napsat jako lineární kombinaci vektorů **báze** $\mathcal{X}$.

Definice 16. Koeficienty lineární kombinace $(\alpha_1, \dots, \alpha_i)$ nazýváme **souřadnice vektoru $\vec{v}$ v bázi $\mathcal{X}$** . Značíme:

$$[\vec{v}]_{\mathcal{X}} = \begin{pmatrix} \alpha_i \\ \vdots \\ \alpha_n \end{pmatrix}$$

Navíc označíme $\forall i \in \nu : \underline{x}^i(v) = \alpha_i$. Zobrazení $\underline{x}^i$ nazveme **i-tý souřadnicový funkcionál v bázi $\mathcal{X}$** . Čísla $\underline{x}^i(\vec{v})$, nazýváme **i-tá souřadnice vektoru $\vec{v}$ v bázi $\mathcal{X}$** .

Věta 17. *Souřadnice v bázi jsou dány jednoznačně.*

Důkaz. Použijeme důkaz sporem. Předpokládejme, že existují alespoň 2 různé lineární kombinace, jejichž výsledkem je vektor $\vec{v}$, tj.

$$\vec{v} = \sum_{i=1}^n \alpha_i \vec{x}_i = \sum_{i=1}^n \beta_i \vec{x}_i \text{ a přitom } (\exists i \in \nu)(\alpha_i \neq \beta_i). \quad (1.3)$$

Dosazením do rovnice (1.3) dostaneme

$$\left(\sum_{i=1}^n \alpha_i \vec{x}_i - \sum_{i=1}^n \beta_i \vec{x}_i = \vec{0} \right),$$

což upravíme na

$$\left(\sum_{i=1}^n (\alpha_i - \beta_i) \vec{x}_i = \vec{0} \right).$$

Tento vztah představuje lineární kombinaci lineárně nezávislých vektorů, jejímž výsledkem je nulový vektor $\vec{0}$. Proto musí jít o triviální lineární kombinaci, neboli $(\forall i \in \nu)(\alpha_i - \beta_i = 0)$, což je spor s předpokladem. $\square$

1.3 Lineární zobrazení

Pojem zobrazení jsme si zavedli již v kapitole 1.1.2.

Definice 18. Mějme 2 vektorové prostory V_1, V_2 nad stejným tělesem T . Zobrazení $A : V_1 \rightarrow V_2$ nazýváme **lineární**, právě když platí

$$(\forall \vec{v}_1, \vec{v}_2 \in V, \forall \alpha \in T) : A(\alpha \vec{v}_1 + \vec{v}_2) = \alpha A(\vec{v}_1) + A(\vec{v}_2). \quad (1.4)$$

Typicky značíme $A\vec{v} := A(\vec{v})$ a toto nazýváme **obraz vektoru $\vec{v}$ při zobrazení A** . Množina všech **lineárních zobrazení** z prostoru V_1 do V_2 se značí $\mathcal{L}(V_1, V_2)$.

Důsledek. Pro $\alpha = 1 : A(1 \times \vec{v}_1 + \vec{v}_2) = A(\vec{v}_1) + A(\vec{v}_2)$. Pro $\vec{v}_2 = \vec{0} : A(\alpha \vec{v}_1 + \vec{0}) = \alpha A\vec{v}_1 + A\vec{0}$. Otázkou je, jestli $A\vec{0} = \vec{0}$? To můžeme zjistit ze vztahu

$$A\vec{v}_1 = A(\vec{v}_1 + \vec{0}) = A\vec{v}_1 + A\vec{0}.$$

Používáme asociativní zákon, který říká, že obraz součtu je roven součtu obrazů. Jediný případ, kdy tato rovnost platí, je pro $A\vec{0} = \vec{0}$.

Poznámka 19. $\mathcal{L}(V_1, V_2)$ je vektorový prostor.

Věta 20. $A \in \mathcal{L}(V_1, V_2) \implies A \left(\sum_{i=1}^n \alpha_i \vec{v}_i \right) = \sum_{i=1}^n \alpha_i A \vec{v}_i$

Důkaz. Použijeme důkaz matematickou indukcí. Začneme prvním krokem, pro $n = 1$. Z (1.4) víme, že $A(\alpha \vec{v}_1 + \vec{v}_2) = \alpha A \vec{v}_1 + A \vec{v}_2$, takže pro $\vec{v}_2 = \theta$ platí $A(\alpha_1 \vec{v}_1) = \alpha_1 A \vec{v}_1$. Následuje indukční krok. Předpokládáme, že platí

$$A \left(\sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right) = \sum_{j=1}^{n-1} \alpha_j A \vec{v}_j$$

pak

$$A \left(\sum_{j=1}^n \alpha_j \vec{v}_j \right) = A \left(\alpha_n \vec{v}_n + \sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right),$$

což není nic jiného než (1.4). Tudíž platí

$$A \left(\alpha_n \vec{v}_n + \sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right) = \alpha_n A \vec{v}_n + A \left(\sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right).$$

Z našeho indukčního předpokladu platí

$$A \left(\sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right) = \sum_{j=1}^{n-1} \alpha_j A \vec{v}_j,$$

tudíž

$$\alpha_n A \vec{v}_n + A \left(\sum_{j=1}^{n-1} \alpha_j \vec{v}_j \right) = \alpha_n A \vec{v}_n + \sum_{j=1}^{n-1} \alpha_j A \vec{v}_j = \sum_{i=1}^n \alpha_i A \vec{v}_i.$$

□

Věta 21. *Mějme číselné těleso T . Pak T nad T je vektorový prostor dimenze 1.*

Příklad. $\mathbb{R}$ nad $\mathbb{R}$ je přímka, číselná osa. ($\mathbb{R}$ v tomto případě značí množinu reálných čísel, nikoli maticí)

Definice 22. Lineární zobrazení $\varphi : V \longrightarrow T$ se nazývá **lineární funkcionál na V** .

Poznámka 23. Množina všech **lineárních funkcionalů** na V je vektorový prostor, nazýván **prostorem duálním** k prostoru V a značí se $V^\# (= \mathcal{L}(V, T))$.

Příklad. Připomeňme si **souřadnicový funkcional** zavedený v sekci 1.2.6.

$(\forall i \in \nu)(\underline{x}^i : V \mapsto T)$ je lineární zobrazení z vektorového prostoru V do číselného tělesa T , tudíž $\underline{x}^i(\vec{x})$ jsou čísla.

Důkaz. Předpokládejme $\vec{v} = \sum_{i=1}^n \alpha_i \vec{x}_i$ a $\vec{w} = \sum_{i=1}^n \beta_i \vec{x}_i$. Pak

$$\alpha \vec{v} + \vec{w} = \alpha \sum_{i=1}^n \alpha_i \vec{x}_i + \sum_{i=1}^n \beta_i \vec{x}_i = \sum_{i=1}^n (\alpha \alpha_i + \beta_i) \vec{x}_i. \quad (1.5)$$

Čísla α_i a β_i získáme použitím souřadnicového funkcionalu na vektory $\vec{v}$ a $\vec{w}$, takže můžeme přepsat jako $\sum_{i=1}^n \underline{x}^i(\alpha \vec{v} + \vec{w})$. Abychom viděli, že souřadnicový funkcional je lineární zobrazení, dosadíme do rovnice (1.5) a dostaneme

$$\sum_{i=1}^n \underline{x}^i(\alpha \vec{v} + \vec{w}) = \alpha \alpha_i + \beta_i = \alpha \underline{x}^i(\vec{v}) + \underline{x}^i(\vec{w}),$$

což je obdoba (1.4), tudíž je souřadnicový funkcional lineární zobrazení. $\square$

1.4 Matice lineárního zobrazení

Matice jsme si zavedli již v sekci 1.1.3.2, nyní je rozebereme více. Nejvíce nás bude zajímat násobení matice s vektorem. Připomeňme si, že vektor z prostoru uspořádaných n -tic $v \in T^{n \times 1}$ je matice o jednom sloupci a n řádcích

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}.$$

K tomu, abychom jej mohli násobit maticí $\mathbb{A}$, musí mít $\mathbb{A}$ n sloupců. Výsledkem bude vektor $\mathbb{A}\vec{v}$ o jednom sloupci a tolika řádcích, kolik měla matice řádků.

Označme $\mathbb{A}_{\bullet j}$ jako j -tý sloupec matice $\mathbb{A}$. Pak $(\mathbb{A}\vec{v})_{i1}$ je i -tý řádek (složka) vektoru $\mathbb{A}\vec{v}$. Jelikož je to jednosloupcový vektor, stačí značit $(\mathbb{A}\vec{v})_i$. Z (1.1)

plyne $\mathbb{A}\vec{v} = \sum_{k=1}^n \vec{v}_k \mathbb{A}_{\bullet k}$, lidsky řečeno je výsledek násobení matice vektorem $(\mathbb{A}\vec{v})$ lineární kombinace sloupců z matice $\mathbb{A}$, kde složky vektoru $\vec{v}$ jsou koeficienty lineární kombinace.

Buď $V = T^n$ s bází $\mathcal{X} = (\vec{x}_1, \dots, \vec{x}_n)$ a buď $W = T^m$. Necht' $\vec{v} = \sum_{i=1}^n \alpha_i \vec{x}_i \in V$ a $A \in \mathcal{L}(V, W)$. Z linearity zobrazení A dle vztahu (1.4) vyplývá

$$A\vec{v} = A \left(\sum_{i=1}^n \alpha_i \vec{x}_i \right) = \sum_{i=1}^n \alpha_i A\vec{x}_i$$

Z tohoto vidíme, že obraz vektoru $\vec{v}$ je lineární kombinace **obrazů bázových vektorů při zobrazení A** . Tyto obrazy bázových vektorů lze uspořádat do matice $\mathbb{A}$:

$$\mathbb{A} = \left(\begin{array}{c|c|c|c} | & | & & | \\ \hline \mathbb{A}_{\bullet 1} & \mathbb{A}_{\bullet 2} & \dots & \mathbb{A}_{\bullet n} \\ \hline | & | & & | \end{array} \right) = \left(\begin{array}{c|c|c|c} | & | & & | \\ \hline A\vec{x}_1 & A\vec{x}_2 & \dots & A\vec{x}_n \\ \hline | & | & & | \end{array} \right) \quad (1.6)$$

Tuto matici označíme ${}^{\mathcal{X}}\mathbb{A}^{\mathcal{E}}$ a čteme „matice zobrazení A v bázích $\mathcal{X}$ a $\mathcal{E}$ “. $\mathcal{E}$ je standardní báze, kterou zavádíme v sekci 1.2.5. Nyní umíme obraz vektoru $\vec{v}$ při zobrazení A napsat jako

$$A\vec{v} = {}^{\mathcal{X}}\mathbb{A}^{\mathcal{E}} [\vec{v}]_{\mathcal{X}}.$$

Pro speciální případ, kdy $\mathcal{X} = \mathcal{E}$, se ${}^{\mathcal{X}}\mathbb{A}^{\mathcal{E}} = {}^{\mathcal{E}}\mathbb{A}^{\mathcal{E}}$, můžeme značit pouze $\mathbb{A}[\vec{v}]_{\mathcal{E}} = \mathbb{A}\vec{v}$.

1.4.1 Matice lineárního zobrazení pro obecné vektorové prostory

Buď V vektorový prostor s bází $\mathcal{X} = (\vec{x}_1, \dots, \vec{x}_n)$ a buď W vektorový prostor s bází $\mathcal{Y} = (\vec{y}_1, \dots, \vec{y}_m)$. Buď $A \in \mathcal{L}(V, W)$. Buď $\vec{v} = \sum_{i=1}^n \alpha_i \vec{x}_i$, pak

$$A\vec{v} = A \left(\sum_{i=1}^n \alpha_i \vec{x}_i \right) = \sum_{i=1}^n \alpha_i A\vec{x}_i.$$

Obrazy $A\vec{v}$ a $A\vec{x}_i$ jsou elementy prostoru W . Z linearity souřadnicového funkcionálu plyne:

$$[A\vec{v}]_{\mathcal{Y}} = [A \sum_{i=1}^n \alpha_i \vec{x}_i]_{\mathcal{Y}} = \sum_{i=1}^n \alpha_i [A\vec{x}_i]_{\mathcal{Y}} = {}^{\mathcal{X}}\mathbb{A}^{\mathcal{Y}} [\vec{v}]_{\mathcal{X}}$$

Kde

$$\mathcal{X}_{\mathbb{A}}^{\mathcal{Y}} = \begin{pmatrix} \left| \begin{array}{c} [A\vec{x}_1]_{\mathcal{Y}} \\ \vdots \end{array} \right| & \dots & \left| \begin{array}{c} [A\vec{x}_n]_{\mathcal{Y}} \\ \vdots \end{array} \right| \end{pmatrix}.$$

Matice $\mathcal{X}_{\mathbb{A}}^{\mathcal{Y}} \in \mathbb{R}^{m \times n}$ a má m řádků a n sloupců.

1.4.1.1 Speciální případ pro 2 obecné shodné prostory

Bud' $V = W$ a necht' $\mathcal{Y} = \mathcal{X} = (\vec{x}_1, \dots, \vec{x}_n)$. Pak označíme

$$\mathcal{X}_{\mathbb{A}} = \begin{pmatrix} \left| \begin{array}{c} [A\vec{x}]_{\mathcal{X}} \\ \vdots \end{array} \right| & \dots & \left| \begin{array}{c} [A\vec{x}_n]_{\mathcal{X}} \\ \vdots \end{array} \right| \end{pmatrix}$$

a $\mathcal{X}_{\mathbb{A}} \in \mathbb{R}^{n \times n}$, takže to bude čtvercová matice rozměru $n \times n$.

1.4.2 Rotace vektorů ve 2D

1.4.2.1 Matice rotace

Mějme zobrazení R , které libovolný vektor v ploše rotuje o úhel φ proti směru hodinových ručiček. Zobrazení R je zřejmě lineární, jak můžeme vidět z obrázku 1.1. To znamená, že pro něj platí $R(\alpha\vec{v}_1 + \vec{v}_2) = \alpha R\vec{v}_1 + R\vec{v}_2$, tedy, že dostaneme stejný výsledek bez ohledu na to, jestli nejprve vektory sečteme a výsledný rotujeme, nebo rotujeme a poté sečteme.

Víme, že lineární zobrazení můžeme vyjádřit maticí. Matici zobrazení R získáme rotací standardní báze.

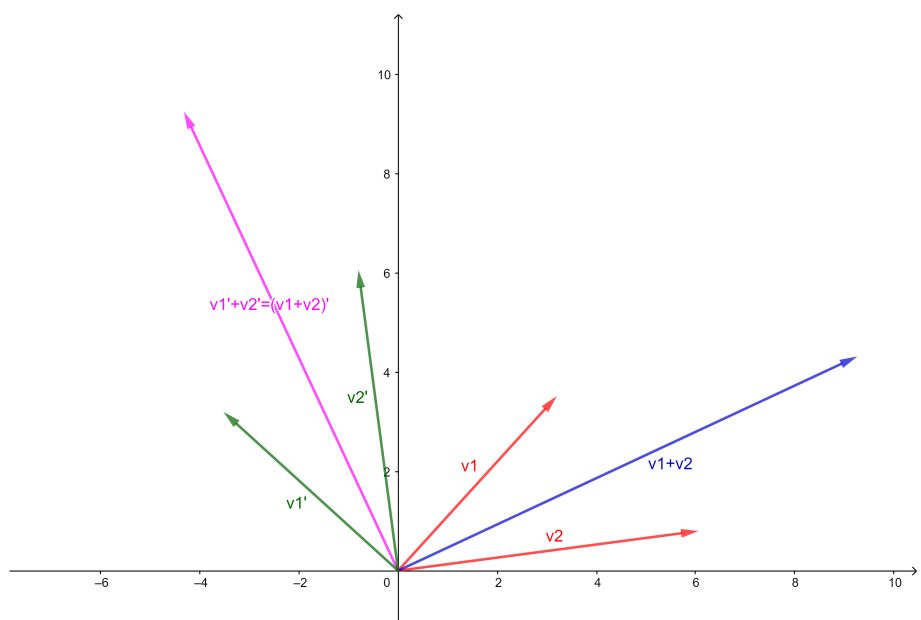
Bud' $\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ a $\vec{e}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ bázi 2D prostoru. Budeme rotovat tyto na sebe kolmé jednotkové vektory o úhel φ . Rotované vektory označíme $\vec{e}'_1$ a $\vec{e}'_2$. Jejich složky vypočítáme následovně:

Ze obrázku 1.2 vidíme, že $\vec{e}'_1$ tvoří odvěsnu trojúhelníka o stranách x -ové složky vektoru $\vec{e}'_1$ a y -ové složky vektoru $\vec{e}'_1$. Víme, že $|\vec{e}'_1| = 1$ a $\cos \varphi = \frac{x}{|\vec{e}'_1|} = x = \cos \varphi$. Obdobně y -ovou složku spočítáme jako $\sin \varphi = y$. Obdobně učiníme pro vektor $\vec{e}'_2$. Úhledně zapsané dostaneme

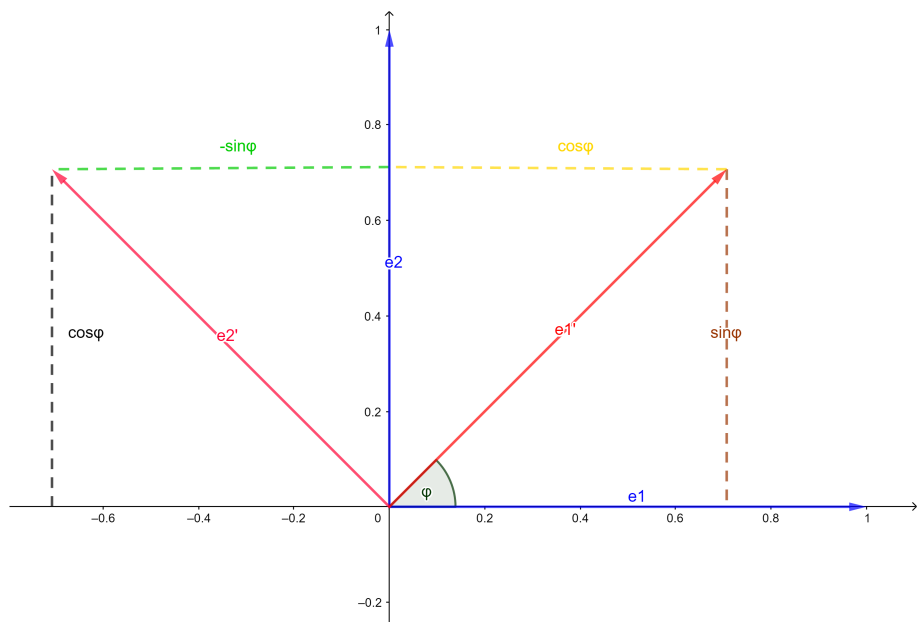
$$\vec{e}'_1 = \begin{pmatrix} \cos \varphi \\ \sin \varphi \end{pmatrix}, \vec{e}'_2 = \begin{pmatrix} -\sin \varphi \\ \cos \varphi \end{pmatrix}.$$

Ze vztahu (1.6) víme, že obrazy báze vektorů tvoří matici zobrazení R . V tomto případě

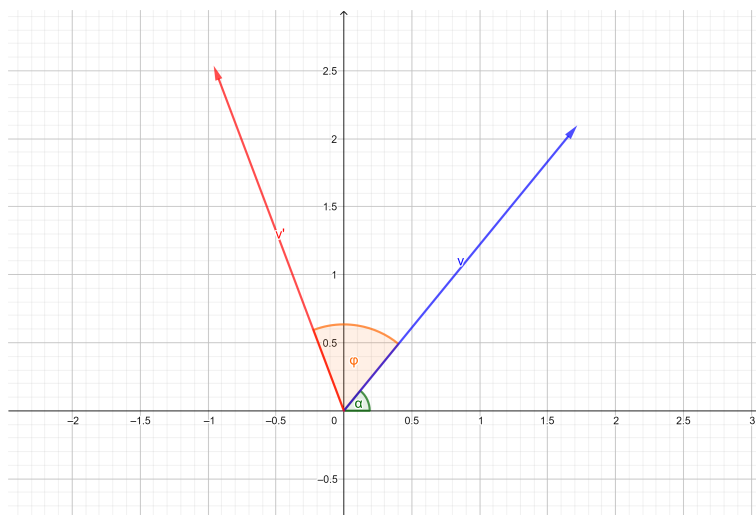
$$\mathbb{R} = \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix}.$$



Obrázek 1.1: Linearita zobrazení R



Obrázek 1.2: Rotace báзовých vektorů o úhel φ



Obrázek 1.3: Rotace vektoru $\vec{v}$ ve 2D

1.4.2.2 Součtové vzorce pro goniometrické funkce

Buď $\vec{v} \in V(\dim V = 2)$. Vektor $\vec{v}$ má velikost r a složky $\begin{pmatrix} x \\ y \end{pmatrix}$. Podobně jako v předchozím příkladu můžeme souřadnice vyjádřit jako součin velikosti a odpovídající goniometrické funkce. Pro obecný vektor platí

$$\vec{v} = \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} r \cos \alpha \\ r \sin \alpha \end{pmatrix}. \quad (1.7)$$

Po otočení o úhel φ dostaneme vektor $\vec{v}'$, který lze vyjádřit jako

$$\vec{v}' = \begin{pmatrix} r \cos(\alpha + \varphi) \\ r \sin(\alpha + \varphi) \end{pmatrix}. \quad (1.8)$$

Stejný výsledek dostaneme, když vynásobíme vektor $\vec{v}$ maticí rotace $\mathbb{R}$.

$$\vec{v}' = \mathbb{R}\vec{v} = \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix} \begin{pmatrix} r \cos \alpha \\ r \sin \alpha \end{pmatrix} = \begin{pmatrix} r(\cos \alpha \cos \varphi - \sin \alpha \sin \varphi) \\ r(\sin \alpha \cos \varphi + \cos \alpha \sin \varphi) \end{pmatrix} \quad (1.9)$$

Srovnáním (1.9) s (1.8) získáme součtové vzorce pro funkce sinus a cosinus.

$$\cos(\alpha + \varphi) = \cos \alpha \cos \varphi - \sin \alpha \sin \varphi$$

$$\sin(\alpha + \varphi) = \sin \alpha \cos \varphi + \cos \alpha \sin \varphi$$

Kapitola 2

Generování objektů pomocí gramatik

2.1 L-systémy

Definice 24. **Abecedou** V nazveme konečnou množinu znaků. Řetězec znaků $a_1 \dots a_n$ označme ω a nazveme **slovo nad abecedou** V . Hodnota n je **délka slova**. Slovo o délce 0 nazveme **prázdné slovo** a označíme ε . Označme V^* jako množinu všech **slov** nad V a označme V^+ množinu neprázdných slov nad V .

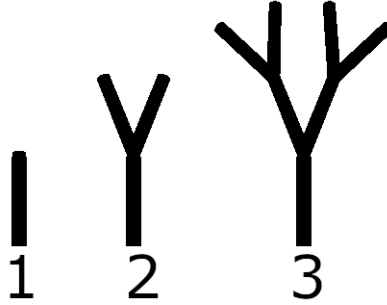
2.1.1 D0L-systémy

Definice 25. Buď V abecedou a ω slovo nad V . Zavedme zobrazení $P : V \rightarrow V^*$ definované jako $P(a) = \omega_a$. To znamená, že každý znak a z abecedy má přiřazené slovo ω_a . Zobrazení P nazveme **gramatikou** na V . Uspořádanou trojici $\langle V, \omega, P \rangle$ (abeceda, **axióm**, gramatika) nazveme **D0L-systém**, neboli deterministický bezkontextový Lindenmayerův systém.[2, s. 4]

Poznámka 26. Tomuto se říká Lindenmayerovy systémy, neboli **L-systémy**. Aristid Lindenmayer tento nápad publikoval roku 1968 jako teorii pro popis rostlin.[6]

Definice 27. Buď $\omega_1 = a_1, \dots, a_n \in V^*$ a $\omega_2 = b_1, \dots, b_m \in V^*$. Zřetězení slov $\omega_1 \omega_2 := a_1 \dots a_n b_1 \dots b_m$ a bude součástí V^* . Buď $\omega_1, \dots, \omega_N \in V^*$. $\omega_1 \omega_2 \dots \omega_N$ definujeme jako $(\omega_1 \dots \omega_{N-1}) \omega_N$.

Poznámka 28. Rovnost $P(a) = \omega_a$ značíme $a \rightarrow \omega_a$ a nazýváme **přepisovací pravidlo**.



Obrázek 2.1: Vývoj objektu v generacích

Definice 29. Mějme D0L-systém $\langle V, \omega, P \rangle$. **Jazykem** nazveme množinu všech slov $V^{\omega, P}$ splňujících:

1. $\omega \in V^{\omega, P}$
2. $\omega = a_1 \dots a_n \in V^{\omega, P} \implies P(a_1) \dots P(a_n) \in V^{\omega, P}$

Poznámka 30. Jazyk vzniká opakovaným aplikováním přepisovacích pravidel na **axióm** ω . V každé **generaci** se na každý znak aplikuje příslušné přepisovací pravidlo a přepis je přidán do nové generace.

Příklad. Mějme **axióm** b a dvě přepisovací pravidla: $a \rightarrow ab$ a $b \rightarrow a[2$, s. 3]. Průběh generování jazyka pomocí přepisovacích pravidel pro 5 generací bude

$$\begin{aligned} g_1 &: b \\ g_2 &: a \\ g_3 &: ab \\ g_4 &: aba \\ g_5 &: abaab \end{aligned}$$

Znaky v D0L-systémech mohou mít i grafickou interpretaci. Například na obrázku 2.1 začínáme se znakem reprezentovaným jednoduchou úsečkou, která je v každé generaci nahrazena znaky reprezentovanými větvením ve tvaru Y. Tento přepis slouží jako příklad, protože pro úplnost je potřeba dodat informace o umístění, směřování, velikosti objektů a jiné. Upřesnění naleznete dále v textu. Na obrázku 2.1 je vidět výsledný objekt po 1, 2 a 3 generacích.

Poznámka 31. D0L-systém nazýváme deterministický, protože pro každý znak existuje právě jedno přepisovací pravidlo. Zobrazení P bývá obvykle definováno výčtem přepisovacích pravidel $a \rightarrow \omega_a$. Ve výčtu se většinou neuvádí přepisy ve tvaru $a \rightarrow a$, tedy pro znaky a , které nemají ve výpisu uvedené přepisovací pravidlo, platí $P(a) = a$.

D0L-systém nazýváme bezkontextový, protože přepisovací pravidla berou v potaz pouze jeden konkrétní znak, ne ty před ani za ním. To je dáno tím, že definiční obor zobrazení $P(a) \rightarrow \omega_a$ je množina $V \subset V^*$, jejíž prvky jsou slova o délce 1.

2.2 Interpretace D0L-systémů pomocí takzvané želví grafiky

Želví grafika je způsob jak interpretovat slova z jazyka $V^{\omega, P}$ do grafického výstupu. Želvu si představme jako objekt, který „leze“ (=pohybuje se) a zanechává za sebou stopu. Ve dvourozměrném prostoru (v ploše) je želva definována svojí pozicí (x, y) , směřováním α (neboli, kam „kouká“) a stavem pera. Stavem pera myslíme, jestli zanechává stopu, jakou barvu má ta stopa a jak tlustá je [2, s. 6]. Želvu v ploše můžeme ovládat následujícími pokyny:

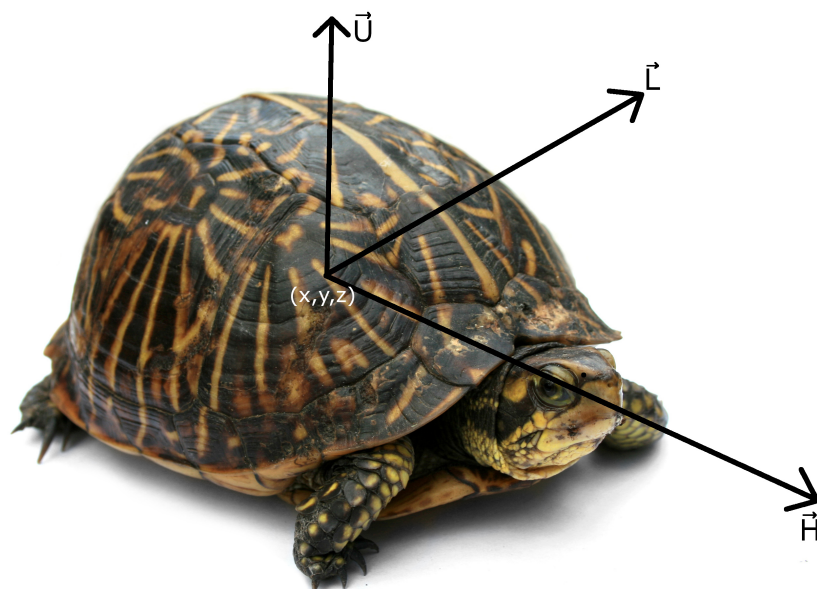
Znak	Příkaz
F	Pohyb želvy vpřed o vzdálenost d .
f	Pohyb želvy vpřed o vzdálenost d bez nakreslení čáry.
+	Rotace doprava (proti směru hodinových ručiček) o úhel φ .
-	Rotace doleva (po směru hodinových ručiček) o úhel φ .

Při pohybu vpřed se z polohy želvy (x, y) stane (x', y') , kde $x' = x + d \cos \alpha$ a $y' = y + d \sin \alpha$. Pokud byl pohyb vpřed vyvolán znakem F, mezi (x, y) a (x', y') se nakreslí čára [2, s. 7].

2.2.1 Větvený D0L-systém

Želva, která pouze leze v ploše, zvládne sice nakreslit krásné fraktály [4, 5, 8], ale pro přiblížení se rostlinným strukturám je potřeba rozšířit želví abecedu V o příkazy pro ukládání polohy a načítání uložených hodnot. Zaveďme tedy:

Znak	Příkaz
[	Ulož aktuální stav želvy do zásobníku.
]	Načti poslední uložený stav ze zásobníku a nastav se na něj.



Obrázek 2.2: Vektory směřování želvy[1]

Díky **závorkám** (díky kterým se tento systém může nazývat „závorkový D0L-systém“) můžeme kreslit **větvené** struktury[2, s. 24]. Příkladem je struktura číslo 3 z obrázku 2.1. Ta vznikla z axiómu F a dvou generací přepisu $F \rightarrow F[+F][-F]$. Výsledné pokyny pro želvu byly:

$$F[+F[+F][-F]][-F[+F][-F]].$$

2.3 Želví grafika ve 3D

2D želví grafika má jistá omezení. Zavedme proto želvu v prostoru.

3D želva je definována svojí pozicí v prostoru, tedy uspořádanou trojicí (x, y, z) , obdobně jako u 2D želvy stavem pera a trojicí na sebe kolmých vektorů, které určují, jak je želva natočená. Vektor $\vec{H}$, neboli heading, znamená směřování, vektor $\vec{L}$, neboli left, jak moc je otočená doleva a vektor $\vec{U}$ - up, jak moc je natočena nahoru [2, s. 18].

Základní příkazy pro 3D želvu jsou:

Znak	Příkaz
+	Rotace kolem osy $\vec{U}$ o úhel α
-	Rotace kolem osy $\vec{U}$ o úhel $-\alpha$
&	Rotace kolem osy $\vec{L}$ o úhel α
$\wedge$	Rotace kolem osy $\vec{L}$ o úhel $-\alpha$
$\backslash$	Rotace kolem osy $\vec{H}$ o úhel α
/	Rotace kolem osy $\vec{H}$ o úhel $-\alpha$
	Otočení o 180° kolem $\vec{U}$
[	Založení větve
]	Ukončení větve
f	Pohyb vpřed o délku d bez zanechání čáry
F	Pohyb vpřed o délku d , mezi bodu se nakreslí čára

2.3.1 Rotace želvy v prostoru

Připomeňme si rotaci vektoru v ploše z kapitoly 1.4.2. Rotace v rovině xy je rotace kolem osy $\vec{z}$, při které se z nemění. Označme

$${}^{\mathcal{E}}\mathbb{R}_{\vec{z}} = \begin{pmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

matici rotace kolem vektoru $\vec{z}$ ve standardní bázi $\mathcal{E} = (\vec{e}_1, \vec{e}_2, \vec{e}_3) = (\vec{x}, \vec{y}, \vec{z})$. Obdobně budou vypadat matice rotace standardní báze v prostoru kolem bázových vektorů $\vec{x}$ a $\vec{y}$:

$${}^{\mathcal{E}}\mathbb{R}_{\vec{y}} = \begin{pmatrix} \cos \alpha & 0 & -\sin \alpha \\ 0 & 1 & 0 \\ \sin \alpha & 0 & \cos \alpha \end{pmatrix}$$

$${}^{\mathcal{E}}\mathbb{R}_{\vec{x}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}$$

Potřebujeme ale vědět, jak bude vypadat matice rotace želví báze. Označme želví bázi $\mathcal{H} = (\vec{H}, \vec{L}, \vec{U})$. Připomeňme, že tyto vektory jsou na sebe kolmé, tak jako bázové vektory standardní báze. Mějme vektor $\vec{v}$.

$$[\vec{v}]_{\mathcal{H}} = \begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} \iff \vec{v} = \alpha \vec{H} + \beta \vec{L} + \gamma \vec{U} \quad (2.1)$$

Rotovaný vektor $\vec{v}$ označme $\vec{v}'$.

$$[\vec{v}']_{\mathcal{H}} = {}^{\mathcal{H}}\mathbb{R}[\vec{v}]_{\mathcal{H}} = {}^{\mathcal{H}}\mathbb{R} \begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} \alpha' \\ \beta' \\ \gamma' \end{pmatrix} \quad (2.2)$$

Souřadnice rotovaného vektoru v bázi $\mathcal{H}$ můžeme vyjádřit jako souřadnice daného vektoru vynásobené maticí rotace. Z tohoto vztahu obdobně jako v (2.1) vyplývá

$$\vec{v}' = \alpha' \vec{H} + \beta' \vec{L} + \gamma' \vec{U} = \begin{pmatrix} | & | & | \\ \vec{H} & \vec{L} & \vec{U} \\ | & | & | \end{pmatrix} \begin{pmatrix} \alpha' \\ \beta' \\ \gamma' \end{pmatrix} \quad (2.3)$$

Dosazením z rovnosti (2.2) do (2.3) získáme

$$\vec{v}' = (\vec{H} \vec{L} \vec{U})^{\mathcal{H}} \mathbb{R} [\vec{v}]_{\mathcal{H}}. \quad (2.4)$$

Toto je vztah pro obecný vektor v želví bázi. Nyní chceme rotovat konkrétně vektory $\vec{H}, \vec{L}, \vec{U}$ a dostat výslednou rotovanou trojici $(\vec{H}' \vec{L}' \vec{U}')$, takže postupně dosadíme za $\vec{v}$. Připomeňme

$$[\vec{H}]_{\mathcal{H}} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, [\vec{L}]_{\mathcal{H}} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, [\vec{U}]_{\mathcal{H}} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

První sloupeček matice $(\vec{H}' \vec{L}' \vec{U}')$ získáme dosazením do (2.4)

$$\vec{H}' = (\vec{H} \vec{L} \vec{U})^{\mathcal{H}} \mathbb{R} [\vec{H}]_{\mathcal{H}} = (\vec{H} \vec{L} \vec{U}) \mathbb{R} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Postupným dosazením zbylých dvou vektorů dostaneme výsledný vztah pro rotaci želví báze:

$$(\vec{H}' \vec{L}' \vec{U}') = (\vec{H} \vec{L} \vec{U}) \mathbb{R}; \mathbb{R} \in \{\mathbb{R}_{\vec{U}}(\alpha), \mathbb{R}_{\vec{L}}(\alpha), \mathbb{R}_{\vec{H}}(\alpha)\}$$

Díky tomu, že vektory $\vec{H} \vec{L} \vec{U}$ jsou na sebe kolmé, budou matice rotace stejné jako matice rotace kolem vektorů standardní báze (v příslušném pořadí) [2, s. 19].

$${}^{\mathcal{E}}\mathbb{R}_{\vec{z}} = {}^{\mathcal{H}}\mathbb{R}_{\vec{U}}(\alpha) = \begin{pmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{aligned}\mathcal{E}\mathbb{R}_{\vec{y}} &= {}^{\mathcal{H}}\mathbb{R}_{\vec{L}}(\alpha) = \begin{pmatrix} \cos \alpha & 0 & -\sin \alpha \\ 0 & 1 & 0 \\ \sin \alpha & 0 & \cos \alpha \end{pmatrix} \\ \mathcal{E}\mathbb{R}_{\vec{x}} &= {}^{\mathcal{H}}\mathbb{R}_{\vec{H}}(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}\end{aligned}$$

Otočení doprava i doleva používá stejnou rotační matici, ale rotace o kladný úhel je proti směru hodinových ručiček (doleva) a rotace o záporný úhel, je po směru hodinových ručiček (doprava). Toto je dáno pořadím vektorů $\vec{H}\vec{L}\vec{U}$ v matici.

2.3.2 Pohyb želvy vpřed v prostoru

Při kroku o délce d vpřed se souřadnice želvy změní z (x, y, z) na (x', y', z') . Mezi body se nakreslí čára.

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} + d \begin{pmatrix} | \\ \vec{H} \\ | \end{pmatrix}$$

Obdobně jako u želvy v prostoru se k určení nových souřadnic používá směřování želvy. Místo úhlu, který v ploše stačil, je směřování želvy dáno složkami vektoru $\vec{H}$.

2.4 Pokročilejší metody pro generování rostlin

Jednoduchými 3D větvenými rostlinami samozřejmě možnosti generování rostlin nekončí. Kromě D0L-systémů existují i **kontextové systémy**, kde přepisovací pravidlo závisí na předchozím a následujícím znaku, takzvaně **levém a pravém kontextu**[2, s. 30]. Další skupinou L-systémů jsou **parametrické systémy**[7].

Parametrické L-systémy mají **gramatiku** definovanou jako uspořádanou čtveřici $G = \langle V, \Sigma, \omega, P \rangle$, kde V je abeceda, neprázdná množina znaků, Σ je množina **formálních parametrů**, ω je neprázdné parametrické slovo - axióm a P množina přepisovacích pravidel. Každý znak slov z parametrických L-systémů má přiřazený žádný, nebo více **formálních parametrů**, značeno $a_1(p_1, p_2, \dots, p_n)a_2(t_1, t_2, \dots, t_m)$. Přepisovací pravidla parametrických L-systémů se značí $a : C \rightarrow \chi$, kdy a je znak, který přepisujeme, C je

podmínka a χ je **parametrické slovo**, na které se znak a přepíše, je-li splněna podmínka. Například aplikujme přepisovací pravidlo $A(t) : t \geq 3 \rightarrow B(t-1)C(t+2)$ na znak $A(5)$ a výsledkem bude $B(4)C(7)$.

Roširit přepis lze i o prvek náhody pomocí **stochastických L-systémů**[2, s. 28], kde mají přepisovací pravidla přiřazenou pravděpodobnost, se kterou je aplikováno. Dalším způsobem, jak generovat věrnější rostliny, je reprezentovat stárnutí rostlin pomocí změny tloušťky stopy, kterou zanechává želva. Například můžeme zvolit přepisovací pravidla a odpovídající tloušťku čáry podle tabulky

Znak	Přepis	Tloušťka čáry
A	$A \rightarrow B$	1
B	$B \rightarrow C$	2
C	$C \rightarrow D$	4
D	$D \rightarrow D / D \rightarrow E \dots$	8...

a grafická interpretace „starších“ částí rostliny (tedy pozdějších písmen v abecedě) bude tlustší, než těch mladších. Výsledky implementace proměnlivé tloušťky čar naleznete v sekci 4.3.

Přidat lze i rostlinné orgány: listy, květy nebo plody.

Vylepšení metod modelování rostlin je celá řada, čímž se otevírají možnosti, jak na tuto práci navázat.

Kapitola 3

Implementace

V této kapitole se seznámíte s implementací D0L-systémů pomocí vícenásobného aplikování přepisovacích pravidel, dále s implementací grafického výstupu pomocí „želví grafiky“. Použil jsem programovací jazyk Python, který se dá číst téměř jako angličtina. Zároveň je kód opatřen mými komentáři, aby byl srozumitelný i bez hlubší znalosti programování. V této kapitole jsou pouze části kódu, na kterých vysvětluji, jak program funguje. Celý kód je přiložen jako příloha k této práci.

3.1 Implementace přepisovacího algoritmu

První program slouží k generaci příkazů pro želvu pomocí přepisovacích pravidel. Na začátku programu definujeme počet generací - proměnná generations, **axióm** - string (textový řetězec) code a množinu přepisovacích pravidel grammar. Pro implementaci přepisovacích pravidel využívám třídu **dictionary** (slovník).

Ta se zakládá: „slovník={}" a může obsahovat vstupy oddělené čárkou ve tvaru <hodnota>:<co chceme přiřadit hodnotě>. Například slovník = {1:„jablko“, 2:„hruška“} obsahuje 2 vstupy: pro hodnoty 1 a 2. Výraz <název slovníku>[klíč] vrátí příslušný odkaz. Například „slovník[1]“ vrátí „jablko“.

Hlavní část programu je for cyklus. For cykly jsou části kódu, které mají pevný počet opakování. V případě tohoto programu je počet opakování dán hodnotou proměnné generations. Každá iterace cyklu reprezentuje generaci D0L-systému. Při každé generaci se vytvoří pomocný textový řetězec new_code, do kterého se zapisují znaky pro další generaci.

Výstup je do textového souboru „input.txt“, který se tak jmenuje, protože slouží jako vstup pro program s implementací želvy.

Výpis kódu 3.1 Přepisovací algoritmus

```
1 generations = 1
2 code = "F"
3 grammar = {"F": "F[+F]F[-F]F"}
4 for i in range(generations):
5     new_code = ""
6     for symbol in code:
7         if symbol in grammar:
8             new_code += grammar[symbol]
9
10        else:
11            new_code += symbol
12
13    code = new_code
14
15 output = open("input.txt", "w")
16 output.write(code)
17 output.close()
```

Tento kód by sice mohl být součástí jednoho programu společně s implementací želvy, ale rozdělení a komunikace pomocí textových dokumentů umožňuje připravit vícero souborů pokynů změnou názvu výstupního souboru.

Nevýhoda tohoto programu je, že se musí přepisovací pravidla ručně psát do slovníku. Někdy jsou přepisovací pravidla dlouhá a složitá, nebo je jich hodně.

3.2 Implementace želví grafiky

Druhý krok v procesu od axiómu po 3D obrázek rostliny je pomocí želvy interpretovat výstupy z přepisovacího programu. Opět jsem programoval v Pythonu. Tento kód je delší, rozdělil jsem jej tedy do několika částí podle funkce.

3.2.1 Vstup

Vstup do programu je načítán z textového souboru „input.txt“, který je výstupem z přepisovacího programu, popsaného v kapitole 3.1.

Výpis kódu 3.2 Načtení vstupu

```
1 with open("input.txt","r") as entry :
2     instructions = entry.read()
```

Výpis kódu 3.3 Konstruktor třídy turtle

```
1 class turtle():
2     def __init__(self,x,y,z,H):
3         self.x = x
4         self.y = y
5         self.z = z
6         self.H = H
```

Na výpisu 3.2 vidíme, jak otevřeme soubor a uložíme jeho obsah do proměnné `instructions`, která je nyní string (textový řetězec). String `instructions` nyní drží všechny pokyny pro želvu.

3.2.2 Želva

3.2.2.1 Reprezentace želvy

Želva je v programu reprezentována třídou `turtle`, kterou jsem naprogramoval. Třídy v Pythonu mohou mít atributy a mohou mít **metody**. Mají **konstruktor** `__init__(self, var1,...)`, který se spustí při zakládání objektu a inicializuje atributy objektu.

Třída želvy v tomto programu má atributy x , y a z určující její pozici v prostoru. Dále má jako atribut matici $\mathbb{H}$, která se skládá ze tří vektorů reprezentujících její orientaci. Kromě těchto parametrů má i 4 metody: rotace kolem jejích tří vektorů, popsaná v kapitole 3.2.2.2, a krok vpřed. Při inicializaci jsou

$$x, y, z = 0 \text{ a } \mathbb{H} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

3.2.2.2 Rotace

Třída želvy má 3 metody umožňující rotaci kolem všech tří vektorů určujících směřování želvy. Funkci rotace spustí hlavní program (o tom více v kapitole 3.2.3), který metodám dá jako parametr úhel α , nebo $-\alpha$, podle

Výpis kódu 3.4 Implementace rotace želvy

```
1      def rot_H(self, a):
2          c = cos(a)
3          s = sin(a)
4          R = matrix((1, 0, 0), (0, c, s), (0, -s, c))
5          self.H = self.H.times(R)
```

toho, jestli jde o rotaci po směru, nebo proti směru hodinových ručiček. Z něj se vytvoří **matice rotace** (viz kapitola 2.3.1), kterou se pak vynásobí matice $\mathbb{H}$ a dostaneme $\mathbb{H}' = \mathbb{H}\mathbb{R}$.

Obdobně jako na výpisu 3.4 vypadají zbylé dvě funkce, mají ale odpovídající rotační matici.

3.2.2.3 Implementace matic

Třída **matrix** je má vlastní implementace matic s vlastnostmi, nutných pro tento program. Třída **matrix** má atribut **seznam** (seřazená n -tice) **sloupčků**, také seřazených n -tic, které dostává **konstruktor**. Dále má následující metody:

- `times(matrix)` - Násobení matic, které se využívá pro rotaci směřování želvy.
- `value(i,j)` - Vrácení hodnoty i, j -tého prvku matice. Tato metoda se využívá jak při násobení matic, tak pro získání hodnoty vektoru $\vec{H}$ při pohybu želvy vpřed.
- `setvalue(i,j,x)` - Nastavení hodnoty i, j -tého prvku matice na hodnotu x . Tato metoda se využívá při násobení matic.
- `print()` - Vypsání matice. Tato metoda je zavedena pro účely **debugingu**, neboli hledání chyb v kódu. Nemá žádnou funkci nutnou pro chod programu.

Implementaci násobení matic naleznete ve výpisu 3.5. Připomněme si vzorec pro i, j -tý prvek součinu matic:

$$(\mathbb{A}\mathbb{B})_{ij} = \sum_{\ell=1}^s \alpha_{i\ell} \beta_{\ell j}.$$

Výpis kódu 3.5 Implementace násobení matic

```
1      def times (self ,M):
2          M2 = matrix ([0 ,0 ,0] ,[0 ,0 ,0] ,[0 ,0 ,0])
3          for i in range (3):
4              for j in range (3):
5                  a=0
6                  for p in range (3):
7                      a+= (self .value (j ,p) *M.
                          value (p ,i) )
8
9                      M2.setvalue (j ,i ,a)
```

Výpis kódu 3.6 Implementace kroku vpřed

```
1  def forward(self ,d ,r):
2      n_x = int (self .x + self .H.value (0 ,0) *d)
3      n_y = int (self .y + self .H.value (1 ,0) *d)
4      n_z = int (self .z + self .H.value (2 ,0) *d)
5      #Výpis do outputového souboru
6      self .x = n_x
7      self .y = n_y
8      self .z = n_z
```

Tuto sumu v Pythonu reprezentuje **for** cyklus, který příslušné hodnoty sečte do proměnné `a`. Poté zavolá funkci `M2.setvalue(i,j,a)`. Připomeňme, že násobení matic není komutativní, takže je relevantní, jestli se násobí takzvaně zprava, nebo zleva. V této implementaci platí

$$\mathbb{A}.\text{times}(\mathbb{B}) = \mathbb{A}\mathbb{B}.$$

3.2.2.4 Implementace pohybu vpřed

Pohyb vpřed je nejdůležitější částí tohoto programu, jelikož při něm zanechává želva stopu - „čáru“. Z kapitoly 2.3.2 víme, že k pohybu vpřed slouží vektor $\vec{H}$. Jeho složky získáme pomocí metody `value()`. Konkrétní kód je ve výpisu 3.6. Jak přesně vypadá výstup, se dočtete v kapitole 3.3.

Výpis kódu 3.7 Implementace větví

```
1 class branch():
2     def __init__(self, x, y, z, V):
3         self.x = x
4         self.y = y
5         self.z = z
6         self.V = V
7
8 branches = []
9
10 if symbol == "[":
11     new_branch = branch(my_turtle.x, my_turtle.y,
12                          my_turtle.z, my_turtle.H)
13     branches.append(new_branch)
14
15 if symbol == "]":
16     a = branches.pop()
17     my_turtle.x = a.x
18     my_turtle.y = a.y
19     my_turtle.z = a.z
20     my_turtle.H = a.V
```

3.2.2.5 Implementace větví

K naprogramování větví jsem vytvořil třídu branch, která má atributy pozice x, y, z a matice $\mathcal{V}$, která drží směřování želvy. Nebylo nutné vytvářet novou třídu, šlo použít třídu turtle. Pro jednoduchost na pochopení jsem zvolil tvorbu nové třídy, aby nám zároveň neexistovalo více želv. Když hlavní program narazí na pokyn založení větve ([), vytvoří nový objekt, pomocí aktuálních atributů želvy a vloží ho do seznamu větví (funkce **append()** přidává vždy na konec seznamu). Při načítání uložené pozice ze seznamu se využije funkce **pop()**, která vyjme poslední prvek seznamu, tedy poslední větev. Definici třídy branch, vytvoření a načtení větve lze vidět ve výpisu 3.7.

Při práci se objevil problém, že výsledné rostliny se negenerovaly správně, byly příliš rovné, protože želva se neotáčela tak, jak měla. Debuging ukázal, že chyba není při rotaci. Po spouště zkoušení, kde by mohla být chyba, se ukázalo, že poslední řádka kódu, který vidíte ve výpisu 3.7, byla „my_turtle.H

Výpis kódu 3.8 Ukázka části kódu hlavního programu

```
1 for symbol in instructions:
2     if symbol in length:
3         my_turtle.forward(length[symbol], width[symbol])
4     if symbol == "+":
5         my_turtle.rot_U(a1)
6     if krok == "-":
7         my_turtle.rot_U(-a1)
8     ...#podmínky pro další znaky
```

= V“, což by za normálních okolností způsobilo chybu, protože „V“ by nebylo definováno. Bohužel jsem jako V označil matici, kterou používám pro konstrukci želvy (viz sekce 3.2.2.1), takže k chybové hlášce nedošlo, ale výsledky byly špatné. A ponaučení? Nepojmenovávat proměnné stejně! Proto se nyní matice želvy jmenuje H a matice větví V.

3.2.3 Hlavní program

Hlavní program je jeden **for cyklus**, který postupně prochází všechny znaky v řetězci instructions. Pro každý znak zkontroluje pomocí funkce **if**, jestli představuje nějaký pokyn pro želvu a pokud ano, vykoná ho. Ukázku kódu vidíte ve výpisu 3.8. Pokud znak nemá přiřazený pokyn, nestane se nic a program pokračuje interpretací dalšího znaku.

3.2.3.1 Implementace různých délek a šířek kroku vpřed

Ve výpisu 3.8 lze vidět, že metoda **forward()** dostává jako vstup parametry length[symbol] a width[symbol]. Toto jsou vstupy ze dvou slovníků. Slovník length má pro každý znak reprezentující pohyb vpřed přiřazenou délku čáry a slovník width zas šířku stopy. Podmínka **if symbol in length** je naplněna, pokud má symbol odkaz ve slovníku length, tedy, že reprezentuje pohyb želvy vpřed.

3.3 3D grafický výstup

PovRay je program pro vytváření 3D grafiky ze zdrojového kódu. V této práci jej používám pro grafický výstup. Zdrojový kód pro PovRay píše želva

při metodě `forward` do textového souboru „output.txt“, který je následně zpracován PovRayem.

3.3.1 Kamera

První věc, která se musí udělat při založení nového PovRay souboru, je definovat kameru. Základní parametry jsou poloha a směřování kamery. Souřadnice jsou zadávány do ostrých závorek. Kameru založíme například jako

```
camera{ location < 0, 0, 100> look_at < 0, 50, 0> }
```

Toto je opravdu naprosté minimum pro založení kamery. Pro rozšířené možnosti kamery nahlédněte do oficiální dokumentace[9].

3.3.1.1 Otáčející se kamera

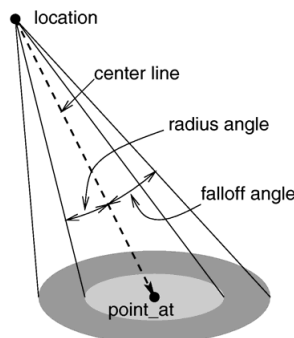
Kameru lze přesouvat, nebo otáčet. Za využití zvláštní funkce **clock** a příkazu **rotate** jsme schopni udělat sérii obrázků, pokaždé s otočenou kamerou. Například `rotate<0,-360*(clock+0.10),0>`. Po té je potřeba použít .ini soubor, ve kterém se specifikuje zdrojový soubor, nastavení proměnné `clock` a počet snímků. Ten vyrendruje scénu několikrát a máme různé pohledy na náš objekt!

```
Input_File_Name="ohnuty_ker.pov"
Initial_Frame=1
Final_Frame=30
Initial_Clock=0
Final_Clock=1
Cyclic_Animation=on
Pause_when_Done=off
```

Takto vypadá soubor „ohnuty_ker.ini“, který třicetkrát vygeneruje scénu ze souboru „ohnuty_ker.pov“. Protože se rotace kamery mění s proměnnou `clock`, bude při každém snímku jiný pohled.

3.3.2 Kreslení čar

Pro náš účel potřebujeme vědět, jak nakreslíme spojnici mezi dvěma body. Nejlepší způsob bude použít válec (anglicky `cylinder`). Ten je definován středem jedné podstavy, středem druhé podstavy a poloměrem podstavy. V PovRayi se objekty definují pomocí příkazu `#declare název = typ objektu {parametry objektu}`. Chceme-li následně objekt vykreslit, stačí zadat jeho název. S tím pak můžeme dále pracovat (například ho posouvat).



Obrázek 3.1: Schéma zdroje světla spotlight[10]

```
#declare line = cylinder {<0,0,0>,<0,15,0>, 2 pigment {color Green}} line
```

Tento kód vytvoří zelený válec od bodu (0, 0, 0) do bodu (0, 15, 0) s poloměrem 2. Vraťme se do kapitoly 3.2.2.4, kde želva píše kód pro PovRay. Kód pro Python bude vypadat takto:

```
1 write = str("#declare_line_cylinder_{<"+x,y,z+>,<"+
    n_x,n_y,n_z+>,"+str(r)+"_pigment_{color_Green}}\
    n_line\n")
2 output.write(write)
```

3.3.3 Osvětlení

Osvětlení vyrendrované PovRay scénu umožňuje lépe vidět, že je objekt trojrozměrný. Po experimentování s **bodovým světlem**, které svítí do všech směrů stejně silně, jsem použil zdroj světla **spotlight**, který svítí v kuželu na určenou souřadnici. U tohoto typu zdroje světla jsou základními hodnotami poloha, barva, bod, na který svítí. Hodnota falloff určuje poloměr podstavy kuželu světla a hodnoty radius a tightness, jak moc světlo slábne dále od středu podstavy. Znázornění těchto hodnot vidíte na obrázku 3.1.

Toto je nastavení **spotlight** zdroje světla ve scéně s rostlinou popsanou v sekci 4.5.

```
light_source {<100, 100, 100> color White spotlight
    radius 60 falloff 10 tightness 10
point_at <0, 200, 0> }
```

Kapitola 4

Výsledky

4.1 Dvojměrná větvená struktura

Rostlina 1 4.1 je ukázkou větveného DOL-systému v ploše. Je převzata z [2]. Axióm je F , jediné přepisovací pravidlo

$$F \rightarrow FF - [-F + F + F] + [+F - F - F]$$

je aplikováno po 4 generace. Úhel rotace je 22.5° .

4.2 Větvená struktura se změnou velikosti úhlu

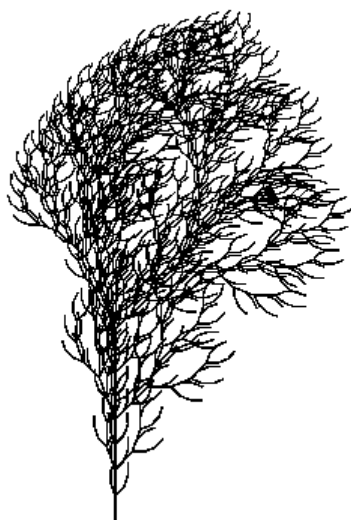
Experimentujme se změnou velikosti úhlu rotace. Použijme rostlinu 2 s axiómem F , přepisovacím pravidlem

$$F \rightarrow F[+F]F[-F][F]$$

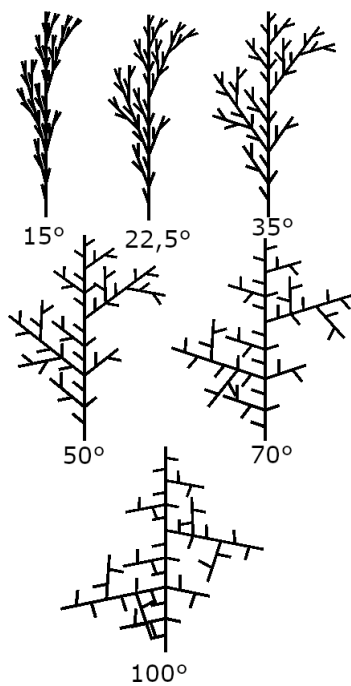
a čtyřmi generacemi. Na obrázku 4.2 lze vidět rostlinu 2 pokaždé s jinou velikostí úhlu.[2]

4.3 Strom s proměnlivou šířkou čáry

Na obrázku 4.3 je vidět rostlina 3, která vznikla použitím principu proměnlivé tloušťky čáry popsaného v sekci 2.4. Mějme axióm A , přepisovací pravidla dle tabulky a 7 generací. Velikost úhlu rotace je 29° .



Obrázek 4.1: Ukázka dvojrozměrné větvené struktury



Obrázek 4.2: Rostlina 2 s rozdílnou velikostí úhlu rotace



Obrázek 4.3: Rostlina 3 - tloustnoucí strom

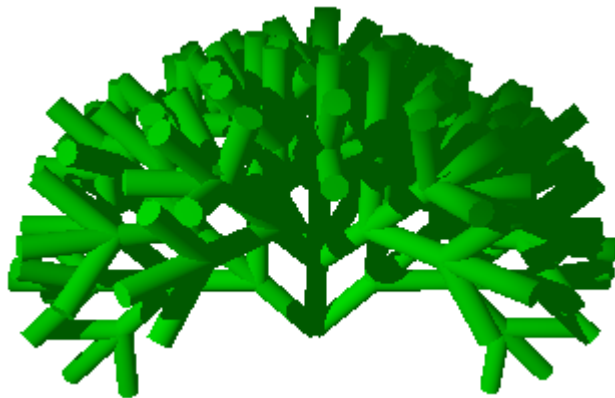
Znak	Přepis	Tloušťka
A	$A \rightarrow B[+A]$	1
B	$B \rightarrow C[-A]$	2
C	$C \rightarrow D[+A]$	2.5
D	$D \rightarrow E[-A]$	3
E	$E \rightarrow F[+A][-A]$	3.5
F	$F \rightarrow F$	4.5

4.4 3D keř

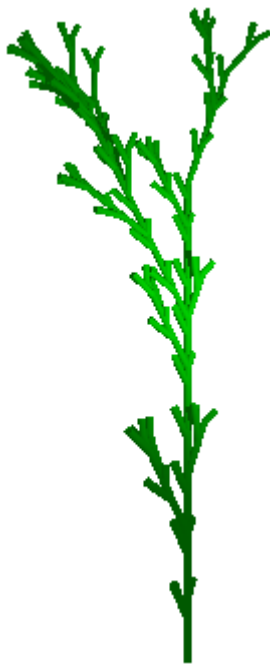
Na obrázku 4.4 vidíte rostlinu 4 převzatou z [5]. Vznikla z axiómu A a přepisovacího pravidla $A \rightarrow [+FA][-FA][\!/FA][!FA]$ po 4 generacích. Velikost úhlu rotace je 45° . Toto je ukázka nejjednodušší prostorové rostliny. Pouze znaky F mají grafickou reprezentaci.

4.5 Větší 3D rostlina

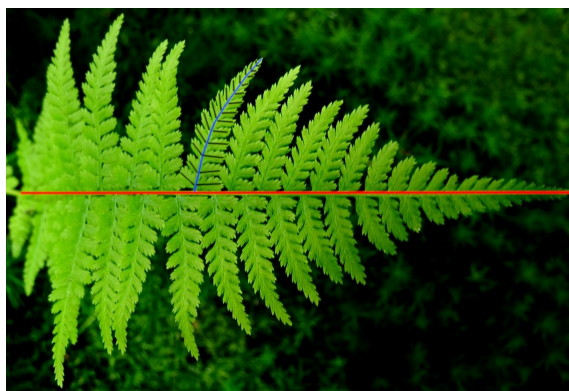
Na obráku 4.5 lze vidět rostlinu vygenerovanou za použití vlastního přepisovacího pravidla $F \rightarrow FF[\wedge + F[\!/F][-F]]F[\&F]$ po 3 generace z axiómu F .



Obrázek 4.4: Rostlina 4 - jednoduchý 3D keř



Obrázek 4.5: Rostlina 5 - větší 3D rostlina



Obrázek 4.6: Kapradí s popsanými částmi

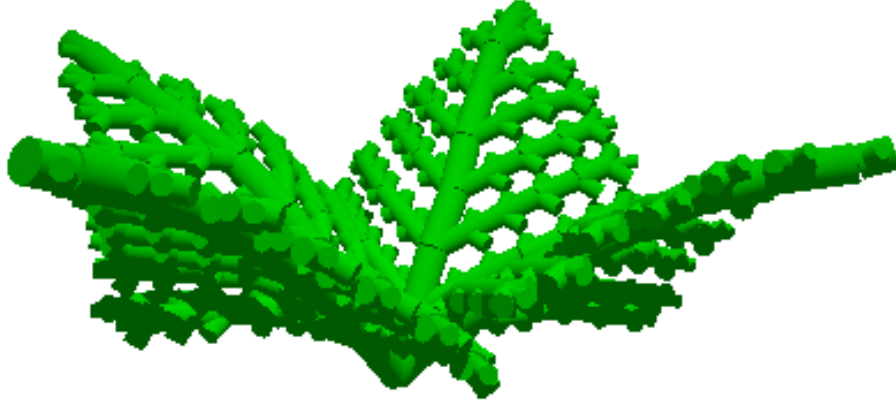
4.6 Kapradí

Rozhodl jsem se si vyzkoušet vymodelování věrného modelu existujícího druhu rostliny. Zvolil jsem si kapradí pro jeho pravidelnost, přišlo mi proto, že bych mohl snadno zvládnout vymyslet přepisovací pravidla pro jeho vymodelování. V této podkapitole vás provedu procesem modelováním kapradí pomocí D0L-systému.

Na obrázku 4.6 je vidět skutečné kapradí s barevně vyznačenými částmi, na které jsem si ho rozdělil. Základní myšlenka byla: v každé generaci vyroste z červeného kmenu modrá větev doprava a doleva. Z modré větve pak vyroste slabá větvička doleva a doprava. Věděl jsem, že nechci pouze jeden stonek, ale více, proto jsem zvolil jako axiom X, který se přepíše na stonky A do různých směrů. Dále jsem chtěl, aby se rostlina ohýbala s délkou stonku dolů, proto jsem přidal znak @, který reprezentoval rotaci kolem osy $\vec{L}$. Ze začátku vypadala přepisovací pravidla následovně:

Znak	Přepis
X	$X \rightarrow [\text{rotace}@A][\text{rotace}@A][\text{rotace}@A]$
A	$A \rightarrow A[++@B][--@B]A$
B	$B \rightarrow B[++@C][--@C]B$
C	$C \rightarrow CC$

Tato přepisovací pravidla negenerovala dostatečný výsledek. Po mnoha hodinách práce jsem si uvědomil, že budu potřebovat nepřepisující se znaky, které neporostou. Zavedl jsem jiný pro hlavní stonek a jiný pro větve, aby později mohly mít jinou délku a tloušťku. Po experimentování s různými



Obrázek 4.7: Rostlina 6 - Kapradí

délkami grafické interpretace znaku C jsem zvolil variantu, ve které se C neprodlužuje. Výsledná tabulka přepisovacích pravidel vypadá takto:

Znak	Přepis	Délka	Tloušťka
X	$X \rightarrow [+@A][@ \div A][@ ++ A][\# \times @A]$	0	0
A	$A \rightarrow @F[+ + B][- - B]A$	3	1,5
B	$B \rightarrow @K[+ + @C][- - @C]B$	3	0,75
C	$C \rightarrow C$	2,5	1
F	$F \rightarrow F$	7	1,75
K	$K \rightarrow K$	4	1,25

Znaky $\#$, $\div$, $\times$ jsou rotace ke správnému nasměrování všech hlavních stonků. F jsou čáry ve hlavním stonku (červená barva na obrázku 4.6) a K jsou modré čáry. Na obrázku 4.7 vidíte výslednou rostlinu po osmi generacích. velikost úhlu rotace $+$, $-$ je 27° a velikost úhlu rotace $@$ je -5° , tedy směrem k zemi. Výsledné délky a tloušťky čar jsou zvoleny na základě experimentací s různými hodnotami.

Celý proces od nápadu, po výsledek, který vidíte na obrázku 4.7 trval přes 6 hodin. Časová náročnost je z velké části dána mojí vlastní nezkušeností v oboru, ale potvrzuje mi to hypotézu, kterou jsem si stanovil na začátku práce, že nalézání přepisovacích pravidel je práce, která obnáší zkušenosti a trpělivost, stejně tak dobře, jako znalosti botaniky a matematiky.

Tento model kapradí nezachycuje skutečnou předlohu zcela věrně, lepších výsledků by šlo dosáhnout například užitím parametrických L-systémů, ale v rámci této práce jsem s výsledkem spokojen.

Závěr

Tato práce pojednává o procesu procedurálního modelování rostlinných struktur pomocí takzvaných D0L-systémů. Podává nutné základy pro pochopení principu procedurálního modelování pomocí formálních gramatik. Práce je shrnutím nutných poznatků pro začátečníka v tomto oboru a otevírá mnohé možnosti pro rozvoj.

Například je možné na této práci stavět a vyzkoušet některé pokročilé možnosti generování rostlin popsaných v podkapitole 2.4. Rozvíjet lze i podobu grafického výstupu přidáním textur čarám nebo implementováním rostlinných orgánů (listů, plodů, květů). Zajímavé je téma odvozování vlastních přepisovacích pravidel pro modelování reálných rostlin. Toto téma by mi přišlo vhodné pro jinou seminární, nebo ročníkovou práci. Aplikace tohoto oboru pak může být například generování krajin v počítačových hrách, simulace růstu rostlin, nebo pokročilé modelování věrných modelů rostlin.

Program pro modelování je jednoduchý, protože implementuje pouze nejjednodušší způsob modelování pomocí D0L-systémů. Přesto je funkční a jsem rád, že tuto část práce jsem zvládl splnit s použitím pouze vlastních implementací želvy a matic. Navíc program funguje na matematických principech popsaných v práci, přestože by to šlo udělat i jinak.

Vlastní práce s programem se ukázala být mnohem složitější, než jsem očekával. Nesnáze dělalo nacházení přepisovacích pravidel i nezkušenost s vykreslovacím programem PovRay. S komplexitou generovaných objektů se ztěžovala práce v PovRayi a problematika toho, jak modely dobře zobrazit. Proto jsem rád, že se mi povedlo dosáhnout vlastního modelu kapradí, který stál velké úsilí, ale pro ukázání výstupů práce je klíčový.

Osobně je pro mě nejdůležitější na celé práci to, že jsem se seznámil se psaním formálních matematických textů, jak co se týče obsahu, tak jak je psát na počítači pomocí typografického systému LaTeX. a editoru LyX. Tyto dovednosti pro mě budou mít velký význam při psaní budoucích prací.

Poděkování

Chtěl bych poděkovat vedoucímu práce panu Ing. Pavlu Strachotovi, Ph.D. za vysokou míru podpory při tvorbě práce, která mi umožnila přesáhnout běžnou úroveň učiva středních škol a naučit se něco nového, nadstavbového a užitečného, co využiji v budoucím životě.

Dále bych chtěl poděkovat škole, že zprostředkovala a umožnila práci u vedoucího mimo pedagogický sbor.

Přílohy

Kód pro přepisovací algoritmus

```
1 generations = 1
2 code = "F"
3 grammar = {"F": "F[+F]F[-F]F"}
4 for i in range(generations):
5     new_code = ""
6     for symbol in code:
7         if symbol in grammar:
8             new_code += grammar[symbol]
9
10        else:
11            new_code += symbol
12
13    code = new_code
14
15 output = open("input.txt", "w")
16 output.write(code)
17 output.close()
```

Kód pro interpretaci D0L-systémů pomocí želví grafiky

```
1 from math import *                                #IMPORT
   KNIHOVNY MATH
2 with open("input.txt", "r") as entry:             #ZPRACOVÁNÍ
   VSTUPU
3     instructions = entry.read()
4 #PŘÍPRAVA OUTPUTU
```

```

5 output = open("output.txt","w")
6 output.write( '#include "colors.inc"\n_background_{color
   _White}\n_camera{ _location_<0.0_,_0.0_,_400.0>\n_
   look_at_<<0.0_,_150.0_,_0.0>\n_rotate_<0,0,0>}_\n_
   light_source_{<100,_100,_100>\n_color_White_\n_
   spotlight_\n_radius_15_\n_falloff_20_\n_tightness_10
   _\n_point_at_<0,_0,_0>\n_}')
7
8 #DEFINICE OBJEKTŮ
9 class matrix():
10     #počítáme od 0,0, do 2,2
11     #i je řádek, j je sloupec
12     def __init__(self,c1,c2,c3):
13         self.columns = [c1,c2,c3]
14
15     def print(self):
16         print(self.value(0,0),self.value(0,1),self.
           value(0,2))
17         print(self.value(1,0),self.value(1,1),self.
           value(1,2))
18         print(self.value(2,0),self.value(2,1),self.
           value(2,2))
19
20     def value (self,i,j):
21         return(self.columns[j][i])
22
23     def setvalue (self,i,j,x):
24         self.columns[j][i] = x
25
26     def times (self,M):
27         #vrátí matici, která je výsledek násobení
           matice s další maticí M
28         M2 = matrix ([0,0,0],[0,0,0],[0,0,0])
29         for i in range (3):
30             for j in range (3):
31                 a=0
32                 for p in range (3):
33                     a+= (self.value(j,p)*M.value(p,i))
34

```



```

35             M2.setvalue(j,i,a)
36
37         return(M2)
38
39     class turtle():
40         def __init__(self,x,y,z,H):
41             self.x = x
42             self.y = y
43             self.z = z
44             self.H = H
45
46         def rot_H(self,a):
47             c = cos(a)
48             s = sin(a)
49             R = matrix((1,0,0),(0,c,s),(0,-s,c))
50             self.H = self.H.times(R)
51             #rotace kolem osy H
52
53         def rot_L(self,a):
54             #rotace kolem osy L
55             c = cos(a)
56             s = sin(a)
57             R = matrix((c,0,s),(0,1,0),(-s,0,c))
58             self.H = self.H.times(R)
59
60         def rot_U(self,a):
61             #rotace kolem osy U = v ploše
62             c = cos(a)
63             s = sin(a)
64             R = matrix((c,-s,0),(s,c,0),(0,0,1))
65             self.H = self.H.times(R)
66
67         def forward(self,d,r):
68             n_x = int(self.x + self.H.value(0,0)*d)
69             n_y = int(self.y + self.H.value(1,0)*d)
70             n_z = int(self.z + self.H.value(2,0)*d)
71             hulp = str(str(self.x)+", "+str(self.y)+", "+str(
                self.z))
72             heelp = str(str(n_x)+", "+str(n_y)+", "+str(n_z))
73             write = str("#declare_line_cylinder_{<"+

```

```

        hulp+">,<"+heelp+">,"+str(r)+"\n\
        pigment_{color_Green}}\n\line\n")
        output.write(write)
74         self.x = n_x
75         self.y = n_y
76         self.z = n_z
77
78     class branch():
79         def __init__(self,x,y,z,V):
80             self.x = x
81             self.y = y
82             self.z = z
83             self.V = V
84
85     #ZAČÁTEK KÓDU
86
87     #INICIALIZACE
88     heading = matrix((1,0,0),(0,1,0),(0,0,1))
89     my_turtle = turtle(0,0,0,heading)
90     my_turtle.rot_U(radians(-90))
91     branches = []
92     al = radians(22.5)
93     length = {"F": 10,"A": 10,"B":15,"C":20,"D":25}
94     width = {"F":2,"A": 1.5,"B":1.75,"C":2,"D":2.5}
95
96     #HLAVNÍ CYKLUS for symbol in instructions:
97         if symbol in length:
98             my_turtle.forward(length[symbol],width[symbol])
99
100         if symbol == "+":
101             my_turtle.rot_U(al)
102
103         if symbol == "-":
104             my_turtle.rot_U(-al)
105
106         if symbol == "&":
107             my_turtle.rot_L(al)
108
109         if symbol == "^":

```

```

110         my_turtle.rot_L(-a1)
111
112     if symbol == "!": #v textu je \, se kterým se
113         python nekamarádi
114         my_turtle.rot_H(a1)
115
116     if symbol == "/":
117         my_turtle.rot_H(-a1)
118
119     if symbol == "|":
120         my_turtle.rot_U(radians(180))#čelem vzad
121
122     if symbol == "[":
123         new_branch = branch(my_turtle.x, my_turtle.y,
124                             my_turtle.z, my_turtle.H)
125         branches.append(new_branch)
126
127     if symbol == "]":
128         a = branches.pop()
129         my_turtle.x = a.x
130         my_turtle.y = a.y
131         my_turtle.z = a.z
132         my_turtle.H = a.V
133
134 output.close()

```

Literatura

- [1] Upraveno, originál pod linencí: I, Jonathan Zander [CC BY-SA (<http://creativecommons.org/licenses/by-sa/3.0/>)]
- [2] PRUSINKIEWICZ, Przemyslaw a Aristid LINDENMAYER. The Algorithmic Beauty of Plants. New York: Springer-Verlag, 1990. ISBN 978-0-387-94676-4. Dostupné také z: <http://algorithmicbotany.org/papers/>
- [3] PYTLÍČEK, Jiří. Lineární algebra a geometrie. Praha: Česká technika - nakladatelství ČVUT, 1997. ISBN 978-90-01-04063-8.
- [4] SMRČKOVÁ, Světlana. Algoritmy pro procedurální modelování rostlin počítačové grafice. Bakalářská práce. České vysoké učení v Praze Fakulta jaderná a fyzikálně inženýrská. Vedoucí práce Ing. Pavel Strachota, Ph.D.
- [5] RUTTKAYOVÁ, Zuzana. Procedural Generation of Plant Models in Computer Graphics. Bakalářská práce. České vysoké učení v Praze Fakulta jaderná a fyzikálně inženýrská. Vedoucí práce Ing. Pavel Strachota, Ph.D.
- [6] LINDENMAYER, Aristid. (1968). Mathematical models for cellular interactions in development I. Filaments with one-sided inputs. Journal of theoretical biology. 18. 280-99. 10.1016/0022-5193(68)90079-9. Dostupné také z: <http://algorithmicbotany.org/papers/>
- [7] HANAN, James Scott. Parametric L-Systems and Their Application to the Modelling and Visualization of Plants. Regina, Saskatchewan, 1992. Disertační práce. Dostupné také z: <http://algorithmicbotany.org/papers/>
- [8] PRUSINKIEWICZ, Przemyslaw a James HANAN. Lindenmayer Systems, Fractals, and Plants. New York: Springer-Verlag Berlin Heidelberg, 1989. ISBN 978-0-387-97092-9.

- [9] POV-Ray: Documentation [online]. [cit. 2020-02-09]. Dostupné z: <https://www.povray.org/documentation/>
- [10] Spotlight scheme. In: POV-Ray: Documentation [online]. [cit. 2020-02-09]. Dostupné z: <https://www.povray.org/documentation/view/3.6.1/310/>